

第 8 章 PCIe 总线的链路训练与电源管理

PCIe 链路的初始化过程较为复杂。PCIe 总线进行链路训练时将初始化 PCIe 设备的物理层、发送接收模块和相关的链路状态信息，当链路训练成功结束后，PCIe 链路两端的设备可以进行正常的数据交换。

链路训练的过程由硬件逻辑完成，而无需系统软件的参与。此外当 PCIe 设备从低功耗状态返回到正常工作模式时，或者 PCIe 链路出现某些错误时，PCIe 链路也需要重新进行链路训练。

8.1 PCIe 链路训练简介

PCIe 总线进行链路训练的主要目的是初始化 PCIe 链路的物理层、端口配置信息、相应的链路状态，并了解链路对端的拓扑结构，以便 PCIe 链路两端的设备进行数据通信。一条 PCIe 总线提供的链路带宽可以是 $\times 1$ 、 $\times 2$ 、 $\times 4$ 、 $\times 8$ 、 $\times 12$ 或者 $\times 16$ ，但是在这个 PCIe 链路上所挂接的 PCIe 设备并不会完全使用这些链路。如一个 $\times 4$ 的 PCIe 设备可能会连接到 $\times 16$ 的 PCIe 链路上。此时该 PCIe 设备在进行链路训练时，必须通知对端链路该设备实际使用的链路状态。

此外 PCIe 总线规定，PCIe 链路两端的设备所使用的 Lane 可以错序进行连接，PCIe 总线规范将该功能称为“Lane Reversal”。在相同的 Lane 上，差分信号的极型也可以错序连接，PCIe 总线规范将该功能称为“Polarity Inversion”。这两种错序连接方式如图 8-1 所示。

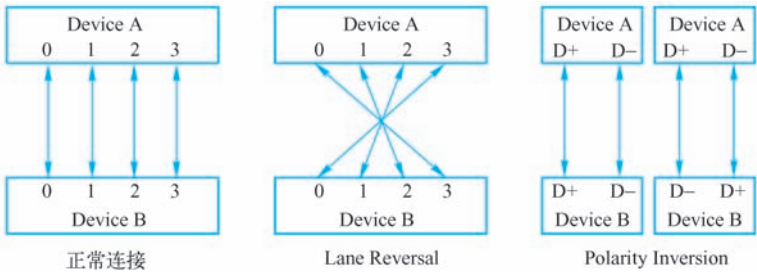


图 8-1 PCIe 设备的错序连接

PCIe 总线提供这些连接方式的主要目的是为了便于 PCB 走线，因为差分信号要求在 PCB 中走线长度相等且间距相等。在一个系统中，如果存在多路差分信号时，PCB 布线较为困难。PCIe 链路允许“Lane Reversal”和“Polarity Inversion”这两个功能，便于 PCB Layout 工程师根据实际情况为差分信号选择更为合理的走线路径，从而降低 PCB 的层数。除了 PCIe 链路，还有许多使用差分信号的串行总线也支持“Lane Reversal”和“Polarity Inversion”这两个功能，但是称呼上有所区别。在一条 PCIe 链路中，可以同时支持“Lane Reversal”和“Polarity Inversion”这两个功能。

PCIe 链路进行链路训练时，需要了解 PCIe 链路两端的连接拓扑结构。一条 PCI 链路可能使用多个 Lane 进行数据交换，而数据报文经过不同 Lane 的延时并不完全相同。PCIe 总线进行链路训练时，需要处理这些不同 Lane 的延时差异，并进行补偿。PCIe 总线规范将这个过程称为 De-skew。

此外 PCIe 总线在链路训练过程中，还需要确定数据传送率。PCIe V1.x 总线使用的数据传送率为 2.5 GT/s，PCIe V2.0 总线使用 5.0 GT/s，而 PCIe V3.0 总线使用 8 GT/s 的数据传送率。当分属不同规范的 PCIe 设备使用同一个 PCIe 链路进行连接时，需要统一数据传送率。如一个 V1.x 的 PCIe 设备与一个 V2.0 的 RC 或者 Switch 连接时，需要将数据传送率统一为 2.5 GT/s。在 PCIe 总线中，如果一个 PCIe 链路的两端分别连接不同类型的 PCIe 设备时，将选择较低的数据传送率。值得注意的是，PCIe 链路在进行初始化时，首先使用 2.5 GT/s 的数据传送率，之后切换到更高的数据传送率，如 5 GT/s 或者 8 GT/s。

在讲述 PCIe 链路训练之前，读者需要了解一些与 Link Number 和 Lane Number 相关的基本概念。在多端口 RC 和 Switch 中具有多个下游端口，而每个端口可以支持×1、×2、×4 等不同宽度的 Lane，如图 8-2 所示。

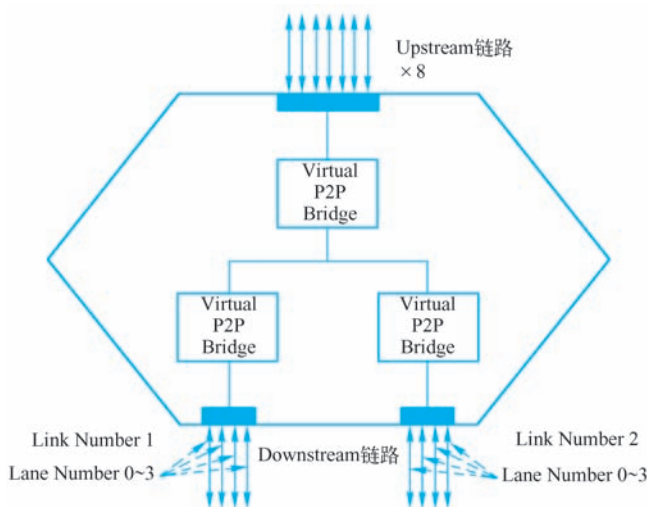


图 8-2 Link Number 和 Lane Number

在一个 Switch 中存在多个下游链路，并使用 0~n 进行编号，其中 $n \leq 255$ 。这些编号保存在 Switch 的硬件逻辑中，而不在 Switch 的配置空间中。这个编号也被称为 Link Number，上图所示的 Switch 中含有两个 Link Number，分别为 1 和 2。

在 Switch 中，还有两类 Lane Number，分别是物理“Lane Number”和逻辑“Lane Number”。其中物理“Lane Number”是链路训练之前使用的 Lane number。一个 PCIe 链路的物理“Lane Number”编号为 0~n，其中 n 为 PCIe 链路的最大 Lane Number，如果一个 PCIe 链路上有 8 个 Lane，则 n 等于 7。

而逻辑“Lane Number”是链路训练结束后使用的 Lane Number。如图 8-1 所示，PCIe 链路允许错序连接，因此物理“Lane Number”与逻辑“Lane Number”并不相同。物理

“Lane Number”与逻辑“Lane Number”的对应关系在链路训练中确定。

除此之外，有些 Switch 支持多种链路配置方式，假设某个 Switch 的下游支持 8 个 Lane。这 8 个 Lane 可以组成 1 个 PCIe 链路，其链路宽度为 8；也可以组成 2 个 PCIe 链路，每个链路宽度为 4；也可以组成 4 个链路，每个链路宽度为 2。

该 Switch 的物理 Lane Number 的编号方法不变，都是从 0~7，但是逻辑 Lane Number 的编号方法将有所区别。如图 8-2 所示的 Switch 中具有 8 个 Lane，组成两个 PCIe 链路，其中每条 PCIe 链路的逻辑 Lane Number 的编号都为 0~3。

PCIe 总线进行链路训练时，需要进行 RC 或者 Switch 的 Link Number 和 Lane Number 的初始化，在第 8.2 节中将详细介绍这些内容。

PCIe 总线进行数据传递时，需要使用时钟进行同步，但是 PCIe 链路并没有提供这个时钟信号，因此在进行链路训练时，接收端需要从发送端的数据报文中提取接收时钟。PCIe 总线规范将这个获得接收时钟的过程称为“Bit Lock”。

在链路训练过程中，PCIe 链路需要首先确定 COM 字符，该字符也标志着链路训练或者链路重训练的开始，PCIe 总线规范将确定 COM 字符的过程称为“Symbol Lock”。如表 7-6 所示，COM 字符为“001111 1010”或者“110000 0101”，该字符为 2 个“0”后 5 个“1”或者 2 个“1”后 5 个“0”，非常便于硬件识别。Bit Lock 和 Symbol Lock 的过程也需要在 PCIe 总线的链路训练中进行。

8.1.1 链路训练使用的字符序列

PCIe 总线进行链路训练时，需要发送一些特殊的字符序列（Ordered-Sets），这些 Ordered-Sets 将在下文中详细介绍，PCIe 总线规范定义了以下几类 Ordered-Sets。有的书籍也将这些 Ordered-Sets 称为 PLP，即物理层报文。

- Training Sequence 1 和 2，简称为 TS1 和 TS2 序列。这两种 PLP 在链路训练的多个状态机中使用，下文将进一步介绍这两种字符序列。
- Idle 序列。在正常情况下，发送端进入 Electrical Idle 状态时，将首先向对端发送若干 Idle 序列，才能进入。Electrical Idle 状态是 PCIe 链路的一个低功耗状态，第 8.1.2 节将详细介绍该状态。
- Fast Training Sequence，简称为 FTS。该字符序列协助接收逻辑获得 Bit/Symbol Lock，接收逻辑需要获得多个 FTS 后，才能确定 Bit/Symbol Lock。
- SKIP 序列。该字符序列的主要作用是进行时钟补偿。

在 PCIe 总线中，字符序列的发送方式与 TLP 和 DLLP 有较大不同。假设一条 PCIe 链路由多个 Lane 组成，那么 TLP 和 DLLP 报文将分散到多个 Lane 中。而字符序列必须同时出现在这些不同的 Lane，这几个 Lane 必须“在同一个时间点”发送字符序列。而不能出现一个 Lane 正在发送这个字符序列进行与链路训练相关的操作，而其他 Lane 进行其他数据传递的情况。PCIe 链路发送 TLP 与发送字符序列的过程如图 8-3 所示。

如在一个×4 的 PCIe 链路中发送 SKIP 序列时，每一个 Lane 中都要出现“COM、SKP、SKP、SKP”这样的数据流。其他字符序列的发送方法也与此类似。而 TLP 或者 DLLP 的发送分散到各个 Lane 上。

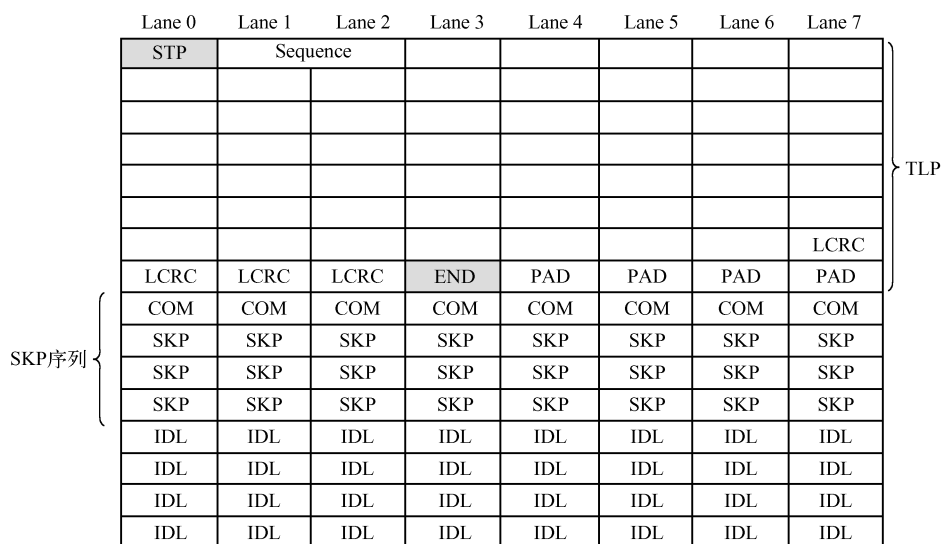


图 8-3 特殊字符序列的发送

1. TS1 和 TS2 序列

在物理层的 LTSSM 状态机中，TS1 和 TS2 序列的使用方法不同。其中 TS1 序列的主要作用是检测 PCIe 链路的配置信息，而 TS2 序列确认 TS1 序列的检测结果。

TS1 和 TS2 序列由 16 个字符组成，单纯从结构上看，TS1 和 TS2 仅仅是第 6~15 个字符的含义不同，但是这两个序列在 LTSSM 状态机中的使用方法不同。TS1 的第 6~15 个字符为 D10.2，而 TS2 的第 6~15 个字符为 D5.2，其中 D10.2 和 D5.2 也是 TS1 和 TS2 的标识号。TS1 和 TS2 序列的其他字符如下所示。

- 第 0 字符为 COM 控制字符，表示 TS1/TS2 序列的开始。TS1/TS2 字符序列将复位 LFSR 寄存器。
- 在链路训练的初始阶段，第 1 字符存放控制字符 PAD，即为空。而在链路的配置阶段，该字符存放端口使用的 Link Number。
- 在链路训练的初始阶段，第 2 字符为控制字符 PAD，即为空。而在链路的配置阶段，该字符存放端口使用的 Lane Number。
- 第 3 个字符为 FTS 序列的个数 (N_FTS)。不同的 PCIe 链路需要使用不同数目 FTS 序列，才能使接收端的 PLL 锁定接收时钟。
- 第 4 个字符存放当前 PCIe 设备支持的数据传送率，第 1 位为 1 表示支持 2.5 GT/s 传送率；第 2 位为 1 表示支持 5 GT/s 传送率；第 3 位为 1 表示支持 8 GT/s 的数据传送率（在 PCIe V3.0 规范中使用）；第 4~5 位保留；第 6 位是一个多功能位，当 PCIe 链路的没有配置成功时可以作为 Notification 位，也可以用作发送链路 De-emphasis 的使能位；第 7 位为 speed_change 位，当该位为 1 时，通知 PCIe 链路对端设备需要改变传送速率。
- 第 5 个字符存放命令。第 0 位为“Hot Reset”，第 1 位为“Disable Link”，第 2 位为“Loopback”，第 3 位为“Disable Scrambling”，第 4 位为“Compliance Receive”。当接收逻辑 RX 收到 TS1 或者 TS2 序列后，将根据该字符的命令进行对应的操作。

2. Idle 序列

在正常情况下，当发送端进入 Electrical Idle 状态之前，必须向对端发送 EIOS 序列。如果 PCIe 设备使用 2.5 GT/s 的传送率时，Idle 序列由 1 个 COM 字符加 3 个 IDL 字符组成，即“COM IDL IDL IDL”；如果 PCIe 设备使用 5 GT/s 的传送率时，Idle 序列由两组这样的字符序列组成，即“COM IDL IDL IDL COM IDL IDL IDL”。Electrical Idle 状态是一种特殊的 Idle 状态，处于该状态时，PCIe 链路使用的功耗最低，该状态的详细解释见第 8.1.2 节。

当发送端退出 IDLE 状态时，必须向对端发送 EIEOS 序列。EIEOS 序列仅在链路传送率大于 2.5 GT 时使用，该序列由 1 个 COM 字符、14 个 EIE 字符和 D10.2 (TS1 识别符) 组成。

PCIe 设备可以根据链路的使用情况确定当前链路是否处于 Electrical Idle 状态，而不是必须收到 Idle 序列后进入该状态。如一个 PCIe 设备在很长一段时间没有收到流量控制报文或者链路处于 Electrical Idle 状态时，也可以推断出对端设备处于 Idle 状态。

3. FTS 序列

单个 FTS 序列由 1 个 COM 字符加 3 个 FTS 字符组成，该序列的主要目的是使接收逻辑 RX 重新获得 Bit/Symbol Lock。发送逻辑需要向对端发送多少个 FTS 序列由接收到的 TS1/2 序列决定，TS1/2 序列的第 3 个字符为需要发送 FTS 序列的个数。

4. SKIP 序列

SKIP 序列由一个 COM 字符加 3 个 SKP 字符组成。物理层提供 SKIP 序列的主要原因是进行时钟补偿。假设一个 PCIe 设备使用的时钟频率为 $2.5\text{ GHz} \pm 300\text{ ppm}$ ，其中 300 ppm 意味着这个时钟源每发出 100 万个时钟可能产生 300 个时钟漂移，即每 3333 个时钟将可能产生一个时钟漂移。如果 PCIe 链路不使用 SKIP 序列，本地时钟与“从报文中提取”的时钟存在的漂移，可能导致数据传送失败。

在 PCIe 设备的接收逻辑 RX 中，使用了两个时钟，一个时钟是通过 PLL 从接收报文中恢复的时钟，另一个时钟是接收逻辑 RX 使用的本地时钟。这两个时钟间的关系如图 8-4 所示。值得注意的是这两个时钟并不完全同步。

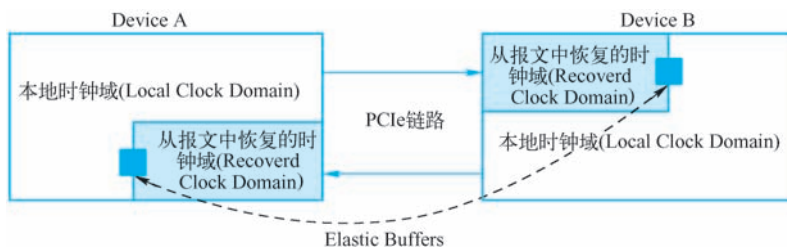


图 8-4 本地时钟域与从报文中恢复的时钟域

在 PCIe 总线中，使用 Elastic Buffer^① 技术处理这两个时钟之间的频率差和相位差。Elastic Buffer 处于本地时钟域与“被恢复的”时钟域之间，由一个同步 FIFO 组成。该 FIFO 的一端使用本地时钟域、而另一端使用“被恢复的”时钟域。其中本地时钟与“被恢复的”时钟频率都为 $2.5\text{ GHz} \pm 300\text{ ppm}$ 。

但是如果 PCIe 设备从数据报文中恢复的时钟频率为 $2.5\text{ GHz} - 300\text{ ppm}$ ，而本地时钟频

① Elastic Buffer 也被称为 Elasticity Buffer 或者 Synchronization Buffer。

率为 2.5 GHz+300 ppm 时, Elastic Buffer 两端的时钟频率并不匹配, Elastic Buffer 将出现 Overrun 的现象;而如果本地时钟频率小于“被恢复的”时钟时, Elastic Buffer 将可能出现 Underrun 的现象。如果 PCIe 总线不采取一些必要的补救措施,那么无论 Elastic Buffer 的容量有多大,都可能出现 Overrun 和 Underrun 的现象。

为此, PCIe 总线规定,物理层的每个 Lane 发送 1180~1538 个字符之后,必须发送一个 SKIP 序列进行时钟补偿。因为在最恶劣的情况下,接收逻辑 RX 每过 1667 个时钟周期,本地时钟就可能与“被恢复的”时钟相差一个时钟周期^①。在 PCIe 总线中,当 Elastic Buffer 收到 SKIP 序列时,可以根据自身的状态选择是增加还是减少 1~2 个 SKIP 序列,从而补偿本地时钟与“被恢复的”时钟之间的频率与相位差。

在一个具体的实现中,可以通过计算,得到 Elastic Buffer 不出现 Overrun 和 Underrun 所需要的最小尺寸。这个最小尺寸与 SKP 序列的发送间隔(多少个时钟周期发送一次 SKP 序列), Max_Payload_Size 参数和 PCIe 链路的数据传送率相关。对此有兴趣的读者可以参考 Elastic Buffer Implementations in PCI Express Devices,以获得详细的量化分析结果,本节对此不做进一步说明。

Elastic Buffer 技术由来已久,除了 PCIe 总线之外,USB 总线、InfiniBand、Fibre Channel 和 Gigabit Ethernet 中也使用该技术处理分属不同时钟域的数据传递。

8.1.2 Electrical Idle 状态

当发送端或者接收端进入 Electrical Idle 状态后,两端的发送逻辑 TX 将驱动 D+和 D-信号的对地电压为相同的值,其值等于 DC 共模电压(DC 共模电压的定义见第 7.3.1 节),从而使发送逻辑 TX 处于“最低功耗”状态。

当发送端处于正常工作模式时,D+和 D-信号差值电压的峰峰值为 $V_{TX-DIFF-PP}$,其值在 0.8 V~1.2 V 之间,而当发送端处于 Electrical Idle 状态时,D+和 D-信号差值电压的峰峰值为 $V_{TX-IDLE-DIFF-DC}$,其值在 0~5 mV 之间。

由此可以发现当发送端处于 Electrical Idle 状态时,在示波器中显示的 D+和 D-信号的对地电压基本相等,此时发送端处于完全“静止”状态,在发送逻辑 TX 中基本没有电流通过,因此从理论上讲处于“最低功耗”状态。

在正常情况下,当发送端或者接收端进入 Electrical Idle 状态之前,都需要使用发送逻辑 TX 向对端发送若干 EIOS 序列,之后才能进入该状态。当发送逻辑 TX 处于 Electrical Idle 状态时,可以处于高阻抗或者低阻抗模式,PCIe 总线规范对此并没有限制。而接收逻辑 RX 处于 Electrical Idle 状态时,其 DC 共模输入阻抗必须在 PCIe 总线规范要求的范围内。

当发送逻辑 TX 进入 Electrical Idle 状态后,至少需要经过 TTX-IDLE-MIN 这段时间延时后,才允许退出该状态,因为对端接收逻辑 RX 的“Electrical Idle Exit detector”部件从启动到正常工作的时间延时为 TTX-IDLE-MIN。如果在“Electrical Idle Exit detector”部件没有正常工作之前,发送逻辑 TX 就退出 Electrical Idle 状态,对端的接收逻辑 RX 将不能检查到这个状态变化,从而导致错误。

① 因为本地时钟与“被恢复的”时钟间可能相差 600 ppm。

发送逻辑 TX 在退出 Electrical Idle 状态时，必须在 $T_{TX-IDLE-TO-DIFF-DATA}$ 时间范围内，需要为 D+ 和 D- 信号提供正常的工作伏值，D+ 和 D- 信号的峰峰值至少为 $V_{TX-DIFF-PP}$ ，其值在 0.8 V~1.2 V 之间。而对端的接收逻辑 RX 发现其 $V_{RX-IDLE-DET-DIFFp-p}$ 的值小于 75 mV 时，将不会退出 Electrical Idle 状态，只有当 $V_{RX-IDLE-DET-DIFFp-p}$ 的值大于 175 mV 时，对端的接收逻辑 RX 才会退出 Electrical Idle 状态。

值得注意的是，发送逻辑 TX 经过发送链路将信号传递到接收逻辑 RX 时，信号将会衰减。虽然发送逻辑 TX 输出的 $V_{TX-DIFF-PP}$ 在 0.8 V~1.2 V 之间，但是接收端收到的 $V_{RX-IDLE-DET-DIFFp-p}$ ，其最小值可能只有 175 mV 左右。

在正常情况下，接收逻辑 RX 在进入 Electrical Idle 状态之前，需要收到发送逻辑 TX 提供的 EIOS 序列，但是有时接收逻辑 RX 在没有收到 EIOS 序列时，发现 $V_{RX-IDLE-DET-DIFFp-p}$ 的值小于 75 mV 时，也可以进入 Electrical Idle 状态。PCIe 总线规范要求接收逻辑 RX 必须在 10 ms 之内判断出当前链路是否处于 Electrical Idle 状态。

接收逻辑 RX 除了可以使用差分电压逻辑，检测当前链路是否处于 Electrical Idle 状态之外，还可以通过其他方式推断出当前链路是否处于 Idle 状态。当 PCIe 链路处于不同的状态时，检测方法有所不同，如表 8-1 所示。值得注意的是，这种推断出的 Idle 状态并不等同于 Electrical Idle 状态。

表 8-1 接收逻辑 RX 推断当前链路是否处于 Idle 状态

链路状态	2.5 GT/s	5.0 GT/s
L0	在 128 μ s 之内，接收逻辑 RX 没有收到 UpdateFC DLLP 或者 SKP 序列时，链路处于 Electrical Idle 状态	
Recovery. RcvrCfg	在 1280 个 UI 之内，接收逻辑 RX 没有收到 TS1 或者 TS2 序列	
Recovery. Speed Successful_speed_negotiation = 1	在 1280 个 UI 内，接收逻辑 RX 没有收到 TS1 或者 TS2 序列	
Recovery. Speed Successful_speed_negotiation = 0	在 2000 个 UI 之内，PCIe 链路没有退出 Electrical Idle 状态 ^①	在 16000 个 UI 之内，PCIe 链路没有退出 Electrical Idle 状态
Loopback. active	在 128 μ s 之内，PCIe 链路没有退出 Electrical Idle 状态	N/A

PCIe 总线通过上表判断而得出的 Idle 状态称为 Logical Idle 状态。在该表中出现的链路状态，如 L0、Loopback. active 等，将在下文详细解释。Electrical Idle 状态是 LTSSM 状态机的基础。Electrical Idle 状态是 PCIe 链路的相对静止状态，使用的功耗较低。当一个 PCIe 设备没有上电，处于复位状态或某些低功耗状态时，其使用的 PCIe 链路将处于 Electrical Idle 状态，读者需要进一步阅读下文的内容以详细了解 Electrical Idle 状态。

8.1.3 Receiver Detect 识别逻辑

Receiver Detect 识别逻辑的主要作用是检测对端的接收逻辑 RX 是否正常工作，Receiver Detect 识别逻辑是发送逻辑 TX 的一部分。PCIe 链路在初始状态时，需要检测对端设备是否

① 没有退出 Electrical Idle 状态指接收逻辑 $V_{RX-IDLE-DET-DIFFp-p}$ 的值小于 75 mV。

存在才能进行链路训练。Receiver Detect 识别逻辑的实现机理是通过检测对端设备接收逻辑的 DC 共模输入阻抗, 来判断接收端是否存在。如果发送逻辑 TX 发现其负载的 DC 阻抗在 Z_{RX-DC} 范围之内或者小于 $40\ \Omega$ 时, 认为对端的接收逻辑 RX 存在。

PCIe 总线规范定义了接收逻辑 RX 在正常工作状态下的 DC 共模输入阻抗 Z_{RX-DC} , 其值在 $40\sim 60\ \Omega$ 范围之内。当接收逻辑 RX 的 V_{cc} 没有上电, V_{D+} 信号或者 V_{D-} 信号的电压伏值大于 0 时, 其 DC 共模输入阻抗 $Z_{RX-HIGH-IMP-DC-POS}$ 最小为 $50\ k\Omega$; 当 V_{D+} 或者 V_{D-} 小于 0 时, 其 DC 共模输入阻抗 $Z_{RX-HIGH-IMP-DC-NEG}$ 最小为 $1.0\ k\Omega$ 。

由此可见当对端接收逻辑 RX 可以正常工作时, 其 DC 共模输入阻抗远小于“没有上电”时的状态。发送逻辑 TX 通过监控 V_{D+} 和 V_{D-} 信号的电压伏值, 可以获得接收逻辑在正常工作状态和 V_{cc} 没有上电时的电流曲线, 从而判断接收逻辑 RX 是否处于正常工作状态。在 PCIe 总线中, 发送逻辑 TX 通过“发送 Detect 序列”判断对端接收逻辑 RX 是否存在。

PCIe 总线发送“Detect 序列”的原理是首先提高 V_{D+} 和 V_{D-} 信号的电压伏值, 然后通过判断对端接收逻辑 RX 的阻抗变化, 识别对端的接收逻辑 RX 是否正常工作。其具体实现方法如下所示。

1) 发送逻辑 TX 在提高 V_{D+} 和 V_{D-} 信号的电压伏值之前, 需要保持 V_{D+} 和 V_{D-} 信号的伏值为一个恒定的 DC 共模电压值。

2) 发送逻辑 TX 暂时提高 V_{D+} 和 V_{D-} 信号的电压, 但是其值不能超过原来共模电压伏值加上 $V_{TX-RCV-DETECT}$ 。此时发送端将产生一个脉冲波形至接收逻辑 RX。这个脉冲波形将穿越发送链路上的 AC 耦合电容, 最后到达接收逻辑 RX。因为接收逻辑 RX 的 Z_{RX-DC} 较小, 因此 $V_{TX-RCV-DETECT}$ 的值不能过大, 否则将在接收逻辑 RX 上产生过大的电流^①, 从而可能损坏接收逻辑 RX。PCIe 总线规定 $V_{TX-RCV-DETECT}$ 的最大值为 $600\ mV$ 。

3) 发送逻辑 TX 根据 V_{D+} 和 V_{D-} 信号的脉冲波形通过接收逻辑 RX 时的电流曲线, 判断接收逻辑 RX 是否正常工作。如果通过这个电流曲线, 发现是 Z_{RX-DC} 起作用, 此时电流强度的有效值较大, 表示接收逻辑 RX 正常工作; 如果发现是 $Z_{RX-HIGH-IMP-DC-POS}$ 起作用, 此时电流强度的有效值较小, 表示接收逻辑 RX 不存在或者没有被加电。

值得注意的是, 在 PCIe V2.x 规范中, 并没有强行规定必须在 V_{D+} 和 V_{D-} 信号上都进行这种 Receiver Detect 测试。在有些实现上, 可能仅使用 V_{D+} 或者 V_{D-} 信号进行这种 Receiver Detect 测试。在 LTSSM 状态机中, 从 Detect 到 Polling 状态的切换时, 需要使用 Receiver Detect 识别逻辑。

8.2 LTSSM 状态机

PCIe 总线在进行链路训练时, 将使用 LTSSM 状态机。LTSSM 状态机由“Detect”“Polling”“Configuration”“Disabled”“Hot Reset”“Loopback”“L0”“L0s”“L1”“L2”和“Recovery”共 11 个状态组成。这些状态分别与 PCIe 总线的链路训练、链路重训练、ASPM (Active State Power Management) 和系统软件的电源管理相关。LTSSM 状态机的转换逻辑如图 8-5 所示, 而各个状态的含义如下所示。

① 在正常情况下, 接收逻辑 RX 的 DC 共模电压为 0, Z_{RX-DC} 虽然较小也不会影响其正常工作。

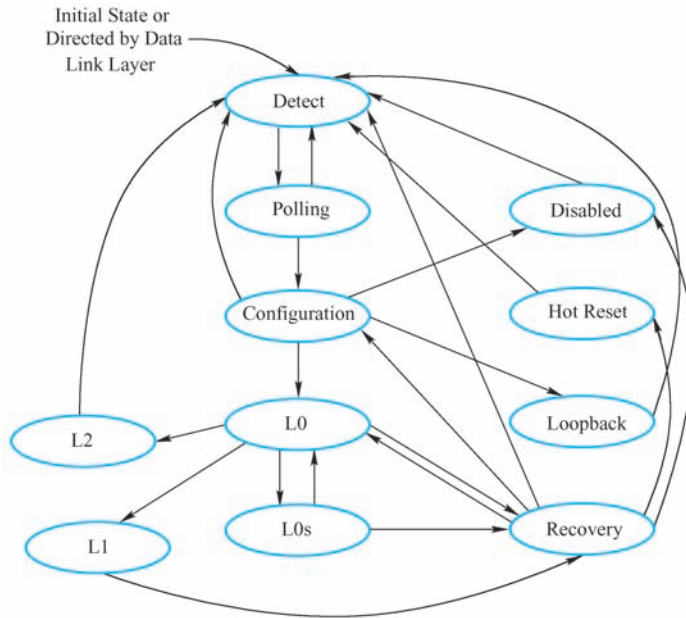


图 8-5 LTSSM 示意图

- Detect 状态。当 PCIe 链路被复位或者数据链路层通过填写某些寄存器后，LTSSM 将进入该状态。该状态也是 LTSSM 的初始状态。当 PCIe 链路处于该状态时，发送逻辑 TX 并不知道对端接收逻辑 RX 的存在，因此需要使用 Receiver Detect 识别逻辑判断对端接收逻辑 RX 是否可以正常工作，之后才能进入其他状态。
- Polling 状态。当 PCIe 链路进入该状态时，将向对端发送 TS1 和 TS2 序列，并接收对端的 TS1 和 TS2 序列，以确定 Bit/Symbol Lock、Lane 的极性。PCIe 链路处于该状态时，将进行 Loopback 测试，确定当前使用的 PCIe 链路可以正常工作。
- Configuration 状态。当 PCIe 链路进入该状态时，将确定 PCIe 链路的宽度、Link Number、Lane reversal、Polarity inversion 和 Lane-to-Lane 的延时。该状态是 LTSSM 状态机最重要的状态。值得注意的是 PCIe 链路在进行初始化时，链路两端统一使用 2.5 GT/s 的数据传送率，直到进入 L0 状态。
- L0 状态。PCIe 链路的正常工作状态。PCIe 链路可以正常发送和接收 TLP、DLLP 和 PLP。PCIe 链路可以从该状态进入 Recovery 状态，以改变数据传送率。
- Recovery 状态。PCIe 链路需要进行链路重训练时需要进入该状态。该状态是 LTSSM 状态机的重要状态。
- L0s、L1 和 L2 状态。PCIe 链路的低功耗状态，其中 PCIe 链路处于 L0s 状态时，使用的功耗相对较高，而处于 L2 状态时，使用的功耗最低。
- Disabled 状态。系统软件可以通过设置寄存器，使 PCIe 链路进入 Disabled 状态。当 PCIe 链路的对端设备被拔出时，LTSSM 也需要进入该状态。
- Loopback 状态。PCIe 链路进入该状态时，发送端口将转发其接收端口接收到的数据，PCIe 测试仪器可以利用该状态进行数据测试。
- Hot Reset 状态。当处理器系统进行 Hot Reset 操作时，PCIe 链路将进入 Recovery 状

态，然后进入 Hot Reset 状态进行 PCIe 链路的重训练。

LTSSM 的各个状态将在下文详细介绍，本章将重点介绍 Detect、Polling、Configuration、L0 和 Recovery 状态，囿于篇幅并不对 Disabled、Loopback 和 Hot Reset 状态进行详细说明，对这些状态有兴趣的读者可以参阅 PCIe 总线规范，以获得进一步信息。

PCIe 设备的物理层进行复位后，LTSSM 状态机首先沿着“Detect”→“Polling”→“Configuration”→“L0”的路径进入到正常工作状态“L0”，这也是链路训练的正常工作路径，也是最重要的一条路径。

物理层链路训练所涉及的内容非常复杂。为便于读者理解，本书仅介绍其主要工作流程，即 PCIe 设备的正常工作路径，以帮助读者掌握链路训练的概要，而不会介绍异常处理和一些并不重要的分支。

8.2.1 Detect 状态

如图 8-6 所示，Detect 状态由 Detect. Quiet、Detect. Active 两个子状态组成。该状态的主要功能是检测 PCIe 链路上是否有 PCIe 设备存在，如果存在，一共使用了多少可用的 Lane 资源。在正常情况下，PCIe 链路将从 Detect 状态迁移到 Polling 状态。



图 8-6 Detect 状态机

当 PCIe 设备进行传统复位操作^①后，首先进入 Detect. Quiet 状态。在多数情况下，如果该 PCIe 设备的对端存在设备时，PCIe 链路的两端将同时进入 Detect. Quiet 状态。因为 PCIe 链路的两端可能同时进行复位操作。

在 PCIe 设备进入 Detect. Quiet 状态时，其发送逻辑 TX 处于“Electrical Idle”状态，此时该设备发送链路的 D+和 D-信号的电压为 DC 共模电压，且为相同的值，此时发送逻辑 TX 使用的功耗最低。

值得注意的是，物理层也可以从 L2、Loopback、Disabled、Polling、Configuration 和 Recovery 状态进入 Detect. Quiet 状态，此时发送逻辑 TX 需要发送必要的 Idle 序列，通知对端设备的接收逻辑 RX，然后经过一段延时后才能进入“Electrical Idle”状态。而在 Detect 状态中，PCIe 设备的发送逻辑 TX 将直接进入“Electrical Idle”状态，并不会使用 Idle 序列通知对端设备的接收逻辑 RX。

当 PCIe 设备处于 Detect. Quiet 状态时，缺省使用 2.5 Gb/s 的数据传送率，即 PCIe V1.x 规定的数据传送率，并置 LinkUp、upconfigure_capable^②等状态位为 0，此时数据链路层处于 DL_Inactive 状态。整个 PCIe 链路处于“完全静止”的状态。

当 PCIe 设备处于 Detect. Quiet 状态超过 12 ms 之后，或者检测到 PCIe 链路上的任何一个

① 不包括 FLR 方式，因为该复位并不影响 LTSSM 状态机。

② upconfigure_capable 状态位在 Configuration 状态收到“Link Upconfigure Capability”为 1 的 TS2 序列后将设置为 1，否则为 0。

Lane 退出“Electrical Idle”状态时，PCIe 设备将进入 Detect.Active 状态。

PCIe 设备进入 Detect.Active 状态后，其发送逻辑 TX 将向该链路的所有“未配置过的 Lane”端发送“Receiver Detection 序列”^①，检测其对端的接收逻辑 RX 是否正常工作。如果所有 Lane 的接收逻辑 RX 都在正常工作时，PCIe 设备进入到 Polling 状态。

如果没有一个 Lane 的接收逻辑 RX 被检测到，PCIe 设备将进入到 Detect.Quiet 状态，此时可能 PCIe 链路的对端没有连接 PCIe 设备，或者该 PCIe 设备并没有正常工作。

如果仅有部分 Lane 正确检测到对端接收逻辑 RX 的存在，物理层将首先等待 12 ms，然后使用该链路的所有“未检测成功过的 Lane”向对端重新发送“Receiver Detection 序列”，进一步识别可用的 Lane。如果这次的检测结果与第一次检测结果相同，物理层将这些“不可用”的 Lane 置为 Electrical Idle 状态，并进入 Polling 状态，如果两次结果不相同，则进入 Detect.Quiet 状态。

这些被标识为“Electrical Idle”状态的 Lane 将不会被 LTSSM 状态机继续使用。有时 PCIe 链路的两端虽然都连接 PCIe 设备，但是这些设备不一定利用了 PCIe 链路上所有的 Lane，下文将举例说明 Detect.Active 状态的运行机制。

假设一个 PCIe 链路由 8 个 Lane 组成，其上游端口与一个 Switch 连接，而下游端口连接一个 EP，如果这个 EP 仅使用了 2 个 Lane。那么第一次 PCIe 链路检测结果为 6 个 Lane 没有与 EP 进行连接，而经过 12 ms 再次进行检测时，可能还检测到有 6 个 Lane 没有与 EP 进行连接，此时该 PCIe 链路认为 EP 仅使用了 2 个 Lane。如果第 2 次检查发现有 7 个或者 5 个 Lane 没有与 EP 进行连接，说明两次检测结果不一致，此时 PCIe 设备将要退回到 Detect.Quiet 状态，并择时重新进入 Detect.Active 状态，重新进行链路探测。

Detect 状态是 PCIe 设备进入的第一个状态，在这个状态中，PCIe 设备需要识别 PCIe 链路的拓扑结构。在这个状态中，物理层使用的检测手段都是基于 PCIe 链路的物理特性，只有 PCIe 设备发现 PCIe 链路的对端具有合法设备后，才能进入 Polling 状态。

8.2.2 Polling 状态

当 PCIe 设备在 Detect 状态中，识别完毕当前链路上可用的 Lane 资源之后，将进入 Polling 状态，Polling 状态由 Polling.Active、Polling.Compliance 和 Polling.Configuration 子状态组成。PCIe 设备可以从 Polling 状态进入到 Configuration 状态，继续进行链路训练，如果 PCIe 设备在 Polling 状态中出现某种错误时，将退回到 Detect 状态，重新进行 PCIe 链路的训练，Polling 状态机的转换逻辑如图 8-7 所示。

如该图所示，PCIe 设备将首先进入 Polling.Active 状态，然后进入 Polling.Configuration 状态，最后退出到 Configuration 状态。

PCIe 设备处于 Polling.Active 状态时，首先检查 Link Control 2 寄存器的“Enter Compliance Bit”位。如果该位为 1，PCIe 设备将进入 Polling.Compliance 状态，值得注意的是，PCIe 链路两端设备的“Enter Compliance Bit”需要被同时置 1，即 PCIe 链路两端的设备需要同时进入到 Polling.Compliance 状态。本节对 Polling.Compliance 状态不做进一步描述。

① Receiver Detection 序列的发送方法见第 8.1.3 节。

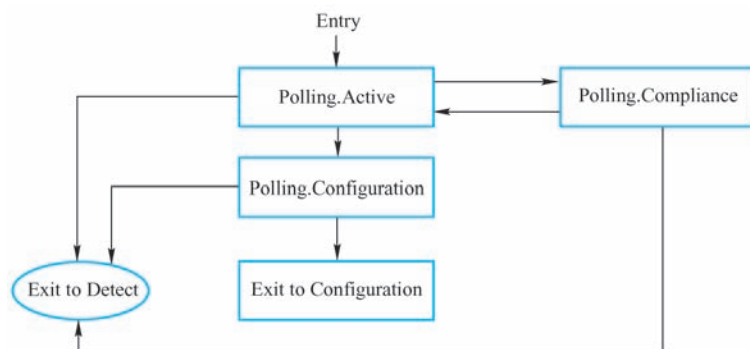


图 8-7 Polling 状态机

如果“Enter Compliance Bit”不为 1，则 PCIe 链路两端设备的发送逻辑 TX 需要向对端^①至少发送 1024 个 TS1 序列，其中 TS1 序列的 Lane/Link Number 必须为“PAD”，即不设置 Lane/Link Number。PCIe 设备使用这些 TS1 序列，获得 Bit/Symbol Lock，这个过程大约需要 64 μ s。值得注意的是，PCIe 链路两端设备退出 Detect 状态时，可能并不完全同步，因此两端设备交换 TS1 序列的过程也并不完全同步。

发送逻辑 TX 在发送 TS1 序列之前，需要保证 D+ 和 D- 信号的 DC 共模电压恢复到正常工作值。因为发送逻辑 TX 在 Detect 状态进行“Receiver Detect”的过程中，曾经将 DC 共模电压提高了 $V_{TX-RCV-DETECT}$ 。

PCIe 设备在发送 1024 个 TS1 序列^②的同时，如果其接收逻辑 RX 从全部“已被正确识别的 Lane”中收到了以下任意一种 8 个连续的报文序列后，该 PCIe 设备将进入 Polling. Configuration 状态。

1) TS1 序列^③，其 Lane/Link Number 为 PAD，而“Compliance Receive”位为 0。

2) TS1 序列，其 Lane/Link Number 为 PAD，而“Loopback”位为 1^④。值得注意的是，发送逻辑 TX 发送“Loopback”位为 1 的 TS1 序列后，PCIe 链路对端设备的接收逻辑 RX 将收到该序列，并将这个 TS1 序列使用内部 Loopback 逻辑直接回送给对端设备（并不是该设备重新生成的 TS1 序列），之后对端设备的接收逻辑 RX 将接收到之前发送逻辑 TX 发送的 TS1 序列。

3) TS2 序列，其 Lane/Link Number 为 PAD。PCIe 链路两端设备进入的 LTSSM 状态机并不一定同步，可能对端 PCIe 设备可能已经进入了 Polling. Configuration 状态，此时该设备将向对端发送 TS2 序列，详见下文。

如果上述条件没有成立，PCIe 设备在经过 20 ms 延时后，判断下列条件。如果这些条件同时成立时，PCIe 设备也将进入 Polling. Configuration 状态。

① 此时发送逻辑 TX 使用在 Detect 状态中，已经正确识别的 Lane 发送这些 TS1 序列。

② 这些 TS1 序列将被链路对端设备回传，因为此时 TS1 序列使用的 Loopback 位为 1。

③ 由于 PCIe 链路支持 Polarity Inversin，因此也可能收到 TS1 序列的补码。

④ PCIe 设备不可能从 Polling 状态进入 Loopback 状态，因此接收逻辑 RX 收到 Loopback 位为 1 的 TS1 序列并不会进入到 LTSSM 的 Loopback 状态。

1) 任何一个“已被正确识别的 Lane”收到了 8 个连续的 TS1 序列, 其中 Lane/Link Number 为 PAD, 而“Compliance Receive”位为 0 或者“Loopback”位为 1; 或者收到 8 个连续的 TS2 序列。而且在收到第一个 TS1 序列之前, 发送逻辑 TX 至少已经发送出 1024 个 TS1 序列。

2) PCIe 设备从 Electrical Idle 状态中退出, 并进入到 Polling.Active 状态时, 所有“已被正确识别的 Lane”至少有一个 Lane 检测到对端设备。

如果该条件也没有成立, PCIe 设备将可能进入 Polling.Compliance 状态。如果 PCIe 链路上任何一个 Lane 的发送链路上连接了一个“对地阻抗为 $50\ \Omega$ ”的电阻后, PCIe 设备也将强制进入 Polling.Compliance 状态。该状态的主要作用是对 PCIe 链路进行检测, 本节对此不做进一步描述。

当 PCIe 设备进入 Polling.Configuration 状态时, 物理层首先处理所有“已识别的”Lane 中, 是否存在极性翻转 (Lane Polarity Inversion) 的现象, 之后进行以下操作。

1) 置 Link Control 2 寄存器的“Transmit Margin”字段为 0b000。

2) 向对端“已识别的 Lane”连续发送 TS2 序列, 其中 Link/Lane Number 为 PAD, 而 Loopback 位为 0。

3) 当收到 8 个连续的 TS2 序列, 而且一共收到 16 个 TS2 序列后, PCIe 设备将进入 Configuration 状态, 否则经过 48 ms 延时后进入 Detect 状态。

当 PCIe 链路两端设备收齐 TS2 序列后, 将基本同步地进入 Configuration 状态。从这个角度来说, TS2 序列是为了同步“异步发送的 TS1 序列”。

在 Polling 状态机中, 还有一个 Polling.Speed 子状态, 该状态的主要作用是调整 PCIe 链路使用的数据传送率。当一个 PCIe 链路两端的设备可以支持高于 2.5 GT/s 的数据传送率时, 可以首先进入该状态, 改变 PCIe 链路的数据传送率。

在许多 PCIe 设备的具体实现中, 并没有使用 Polling.Speed 子状态。此时在 PCIe 链路的训练过程中, 将缺省使用 2.5 GT/s 的数据传送率。此时 LTSSM 状态机将首先沿着 Detect→Polling→Configuration→L0 的路径进入 L0 状态, 并使用 2.5 GT/s 的数据传送率, 即便 PCIe 链路两端的设备都支持更高的数据传送率。

当 PCIe 设备改变数据传送率时, 需要在 L0 状态, 通过向对端设备发送 TS1 序列 (其 speed_change 位为 1), 使 PCIe 链路两端设备进入 Recovery 状态后, 才能改变缺省使用的数据传送率。

8.2.3 Configuration 状态

Configuratoion 状态是 LTSSM 的重要状态, 该状态完成 PCIe 链路的主要配置工作, 包括 Link Number 和 Lane Number 的协商, 并使 PCIe 链路进入正常工作状态 L0。如图 8-8 所示, Configuration 状态由多个子状态组成。

其中 Configuration.Linkwidth.Start 和 Configuration.Linkwidth.Accept 状态判断当前 PCIe 链路的有效宽度; Configuration.Lanenum.Wait 和 Configuration.Lanenum.Accept 状态判断当前

PCIe 链路的物理“Lane Number”与逻辑“Lane Number”的对应关系；而 Configuration.Complete 状态负责处理“Lane-to-Lane de-skew”。由上文所述，一条 PCIe 链路可能由多个 Lane 组成，而使用这几个 Lane 进行数据传递时，可能存在速度差异，即“Skew”，“Lane-to-Lane de-skew”就是消除这个速度差异的方法。

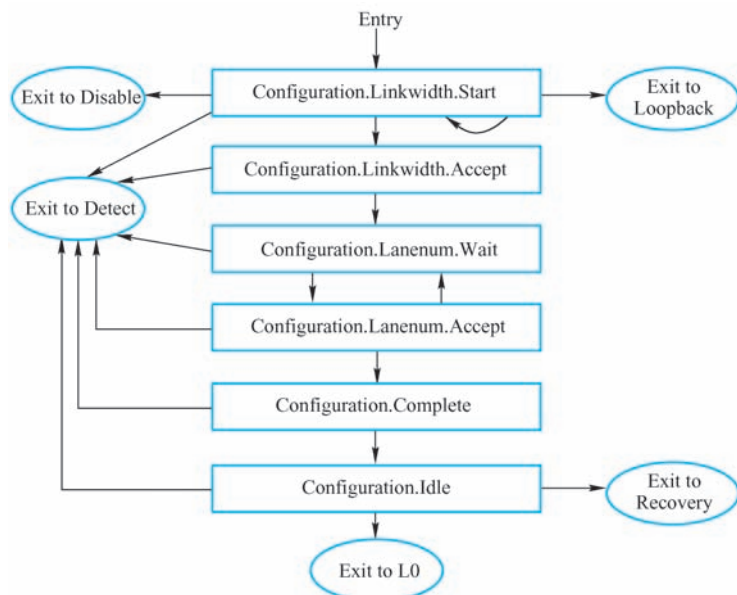


图 8-8 Configuration 状态机

在 LTSSM 状态机中，Polling 状态正常结束时将进入 Configuration 状态。此外当 Recovery 状态出现某些错误，没有正常进入 L0 状态时，也可能首先进入 Configuration 状态，然后经过错误处理之后，重新返回 L0 状态。

在 PCIe 总线中，LTSSM 状态机从 Polling 状态进入 Configuration 状态时 Linkup 位为 0，因为对应 Lane 不曾被激活；而从 Recovery 状态进入该状态时 Linkup 位为 1。进入 Configuration 状态时，PCIe 链路上游端口（包括 RC 端口或者 Switch 的下游端口）Link Status 寄存器的 Link Training 位被硬件置 1，从该状态进入 L0 状态时，该位被清零。

1. Link Number 的协商过程

PCIe 总线使用自协商的方法，确认不同 PCIe 链路的 Link Number。下文以一个具有两个端口的 Switch 为例，简要说明 Link Number 的协商机制，多端口 RC 的 Link Number 协商机制与此相似。该 Switch 与 EP 的连接方法如图 8-9 所示。

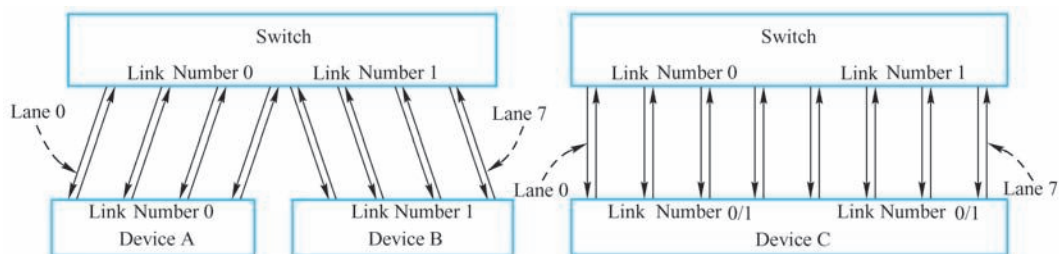


图 8-9 Link Number 的协商过程

在该 Switch 中共有 8 个 Lane，并且可以分解为 2 个 PCIe 链路^①，每个链路的最大宽度为 4，即含有 4 个 Lane，这 4 个 Lane 并不能自由组合形成两个 PCIe 链路，而是第 0~3 个 Lane 组成一个 PCIe 链路，而第 4~7 个 Lane 组成另一个 PCIe 链路。其中每一个 PCIe 链路可以独立使用，连接不同的 EP，这些 EP 最多可以使用 4 个 Lane。当然这 8 个 Lane 也可以合并成一个 PCIe 链路，与一个 EP 连接。

Switch 不会预知其下游链路 with EP 的连接拓扑结构。因此在 Configuration.Linkwidth 状态时，Switch 将向第 0~3 个 Lane 发送 TS1 序列，其 Link Number 为 N；而向第 4~7 个 Lane 发送 TS1 序列，其 Link Number 为 N+1。如图 8-9 所示，Device A 收到的 TS1 序列，其 Link Number 为 N，而 Device B 收到的 TS1 序列，其 Link Number 为 N+1。Device A 或者 B 收到这些 TS1 序列后，将向 Switch 发送 TS1 序列，其 Link Number 为 N 或者 N+1。

而 Device C 收到的 TS1 序列，其 Link Number 为 N 或者 N+1。值得注意的是，此时 Device C 将选择唯一的 Link Number “或者为 N 或者为 N+1” 回传给 Switch，而并不是将 “N 和 N+1” 都回传给 Switch。

Switch 根据收到的 TS1 序列判断其链路的连接拓扑结构，如果 Switch 收到的 TS1 序列中包含两个不同的 Link Number，则表示 8 个 Lane 被分解为两个 PCIe 链路 with 两个 PCIe 设备相连；如果只有一个 Link Number，则表示 8 个 Lane 被合并为 1 个 PCIe 链路 with 1 个 PCIe 设备相连。如果这个 Switch 支持 4 路 PCIe 链路，则 Switch 需要向这 4 个 PCIe 链路发送的 Link Number 为 N~N+3，共 4 种 TS1 序列；如果 Switch 支持 8 路 PCIe 链路，则 Switch 需要向这 8 个 PCIe 链路发送的 Link Number 为 N~N+7，共 8 种 TS1 序列。

PCIe 总线使用自协商的方法识别下游链路的拓扑结构。PCIe 总线在 Configuration 状态中，使用 Configuration.Linkwidth.Start 和 Configuration.Linkwidth.Accept 两个子状态，进行 PCIe 链路两端设备 Link Number 的协商，以确定下游链路 with PCIe 设备的连接拓扑结构。

当 PCIe 链路进入 Configuration.Linkwidth.Start 状态时，RC 端口或者 Switch 的下游端口将向其下游链路发送 TS1 序列以确定 PCIe 链路的 Link Number，下游设备也将会回送 TS1 序列。上文以连接 2 个 EP 的 Switch 为例，说明 Link Number 的自协商过程。RC 端口与其下游设备 Link Number 的协商过程与此类似，本节对此不做进一步说明。

当 PCIe 链路进入 Configuration 状态时，如果 Linkup 位为 0 时，Switch 需要向下游链路的每一个 Lane 都发送若干个 TS1 序列，在这个 TS1 序列中的 Link Number 字段分别为 0 和 1^②，而 Lane Number 字段为 PAD。这里使用的 Link Number 由 Switch 内部由硬件逻辑进行编号，其编码在 0~255 之间，本节使用 0 和 1 作为 Link Number。注意在这个 TS1 序列中，Loopback 位不再为 1，否则对端设备将接收到的 TS1 序列直接 Loopback，回送给发送端。

值得注意的是，如果 Linkup 或者 upconfigure_capable 位为 1 时，即从 Recovery 状态进入该状态时，TS1 序列中的 Link Number 字段为 PAD (K23.7)，此时的处理方法与从 Polling 状态进入该状态有些区别。本节对这种情况不做进一步描述。

Switch 通过发送部件 TX，经过其下游链路发送完毕这些 TS1 序列后，开始监控其接收

① 此时 Switch 中包含 2 个 LTSSM 状态机，也相应具有 2 个物理层、链路层和事务层。

② 前 4 个 Lane 的 Link Number 为 0，而后 4 个 Lane 的 Link Number 为 1。

部件 RX。如果接收部件 RX 的任何一个 Lane 收到至少 1 个来自下游设备的 TS1 序列后（该序列的 Link Number 和 Lane Number 字段都为 PAD），需要进一步判断是否任何一个 Lane 收到 2 个连续的 TS1 序列（该序列的 Link Number 为 0 或者 1，即与发送的 TS1 序列 Link Number 相同，而 Lane Number 为 PAD）。上述检测成功之后，Switch 的下游端口将进入 Configuration.Linkwidth.Accept 状态。

而与 Switch 连接的下游设备在进入 Configuration.Linkwidth.Start 状态时，首先向其上游链路发送若干个 TS1 序列（该序列的 Link 和 Lane Number 字段都为 PAD），这也是 Switch 的下游端口首先收到这个 TS1 序列（Link 和 Lane Number 都为 PAD）的原因。

之后下游设备等待来自 Switch 的 TS1 序列，下游设备可能收到 TS1 序列中的 Link Number 为 0 或者 1（图 8-9 中的 Device C 可能收到这样的 TS1 序列），但是下游设备仅使用其中一个 Link Number（PCIe 总线并没有规定是使用 0 还是 1）组成若干个 TS1 序列，并通过其上游链路发送给 Switch，这个 TS1 序列的 Lane Number 为 PAD。

随后下游设备也进入 Configuration.Linkwidth.Accept 状态，至此在这条 PCIe 链路上的两个设备都将进入 Configuration.Linkwidth.Accept 状态。

在 Configuration.Linkwidth.Accept 状态时，Switch 的下游端口将分析接收到的 TS1 序列，并将所有 Link Number 相同的 Lane 组合在一起，通过此步骤，Switch 的下游端口可以确定下游链路的连接拓扑结构，即下游链路由哪些 Lane 组成。对于图 8-9 所示的实例，Device A 使用 Switch 下游链路的 Lane 0~3，Device B 使用 Switch 下游链路的 Lane 4~7，而 Device C 使用 Switch 下游链路的 Lane 0~7。

Switch 在确定了连接拓扑结构后，将向 PCIe 链路^①的所有 Lane 发送若干个 TS1 序列，之后进入 Configuration.Lanenum.Wait 状态。值得注意的是，在这个 TS1 序列中 Link Number 为确定的值，而 Lane Number 在 0~n-1 之间，此时使用的 Lane Number 为逻辑号。在图 8-9 中，虽然 Device B 使用 Switch 的物理 Lane 为 4~7，但是发向物理 Lane 4~7 的 TS1 序列的 Lane Number 为 0~3。

在进入 Configuration.Linkwidth.Accept 状态时，下游设备等待 Switch 端口的 TS1 序列。如果下游设备收到 2 个连续的 TS1 序列，其 Link Number 等于在 Configuration.Linkwidth.Start 状态发送给 Switch 的 Link Number，而且 Lane Number 不为 PAD 时，下游设备将向 Switch 发送若干个 TS1 序列，之后下游设备进入 Configuration.Lanenum.Wait 状态。

该序列的 Link Number 为接收到的值，而 Lane Number 在 0~m-1^②之间。如果下游设备没有使用错序连接方式，那么下游设备发送的 Lane Number 将与 Switch 的 Lane Number 直接对应；否则错序对应。本节虽然使用了一定篇幅讲述 PCIe 总线 Link Number 的自协商过程，但也仅说明了该过程的一个子集。PCIe 总线的自协商过程较为复杂，相对较难实现。

为此 PLX 公司采用了另外一种 Link Number 的协商机制。下面以 PEX8518 芯片^③为例说明这种协商机制，该芯片并没有采用 PCIe 总线规定的协商机制，而是使用寄存器配置其下

① 该 PCIe 链路可能由部分 Lane 组成，如与 Device A 连接的链路由 Lane 0~3 组成，与 Device B 连接的链路由 Lane 4~7 组成。

② $m \leq n$ ，因为下游设备可能只使用 PCIe 链路的部分 Lane。

③ PEX8518 是 PLX 公司设计的 Switch 芯片。

游端口的使用情况。

PEX8518 芯片的下游链路由 16 个 Lane 组成（分别为 0~15），最多可以支持 5 个端口，分别为 Port0~4。这 5 个端口能够使用的 Lane 由 Port Configuration 寄存器规定，而不是通过 Configuration 状态自适应检测其下游链路的拓扑结构。该寄存器的值与其下游端口的对应关系如表 8-2 所示。

表 8-2 Port Configuration 寄存器与下游端口的对应关系

Port Configuration 寄存器的值	链路宽度				
	端口 0	端口 1	端口 2	端口 3	端口 4
0x00	×4 (0~3)	×4 (4~7)	×4 (8~11)	×4 (12~15)	不使用
0x02	×8 (0~7)	×8 (8~15)	不使用	不使用	不使用
0x03	×8 (0~7)	×4 (8~11)	×4 (12~15)	不使用	不使用
0x04	×8 (0~7)	×4 (8~11)	×2 (12~13)	×2 (14~15)	不使用
0x05	×8 (0~7)	×2 (8~9)	×2 (10~11)	×4 (12~15)	不使用
0x06	×8 (0~7)	×2 (8~9)	×4 (10~13)	×2 (14~15)	不使用
0x08	×8 (0~7)	×2 (8~9)	×2 (10~11)	×2 (12~13)	×2 (14~15)
0x09	×4 (0~3)	×4 (4~7)	×4 (8~11)	×2 (12~13)	×2 (14~15)

由上表所示，PEX8518 芯片使用静态表进行 Link Number 的确认，而没有使用 PCIe 总线规定的自协商机制，从而在保证配置灵活性的同时，极大简化了 Link Number 的协商难度。但是使用这种方法不适用于“下游链路拓扑结构不可预知”的应用。

因此在设计中，尽量避免使用 PEX8518 芯片连接一个 PCIe 插槽，因为这个插槽上的 PCIe 设备使用的 Lane 并不确定，使用静态分配 Lane 的方法并不合适，但是这并不妨碍 PEX8518 芯片在嵌入式领域，尤其在电信领域的大规模应用。

2. Lane Number 的协商过程

物理层在确认 PCIe 链路的 Link Number 之后，开始进行 Lane Number 的协商。PCIe 总线使用 Configuration.Lanenum.Wait 和 Configuration.Lanenum.Accept 两个子状态完成 Lane Number 的协商。下面仍然以图 8-9 为例说明 Lane Number 的协商过程。

与 Link Number 的协商过程相比，Lane Number 的协商过程较为简单。本小节以一个实例说明 Lane Number 的实现过程，而并不深究 LTSSM 状态机的详细迁移过程。

如图 8-1 所示，PCIe 链路允许错序连接，因此 Switch 的下游端口与下游设备的上游端口使用的 Lane Number 并不一定完全一致，因此物理层需要进行 Lane Number 的协商。当 Switch 与 Device A 完成 Link Number 的协商后，将进行 Lane Number 的协商。

首先 Switch 向 Device A 的 4 个 Lane 发送 TS1 序列，在这个 TS1 序列中的 Link Number 字段为 0，而 Lane Number 字段分别为 0、1、2 和 3。

当 Device A 收到这些 TS1 序列后，也将向 Switch 回送使用 TS1 序列，在这个 TS1 序列中的 Link Number 字段为 0，而 Lane Number 字段为 Device A 的物理 Lane Number 号。如果 Switch 与 Device A 使用图 8-1 中图所示的错序连接方法时，Device A 回送的 TS1 序列的 Link Number 字段为 3、2、1 和 0，如果使用图 8-1 左图所示的连接方法时，Device A 回送的 TS1 序列为 0、1、2 和 3。

当 Switch 收到 Device A 的回送 TS1 序列后，可以获得 PCIe 链路的连接信息，从而确定

当前 Lane 的连接拓扑结构，并完成逻辑 Lane Number 与物理 Lane Number 的对应关系。当使用图 8-1 右图所示的错序连接方法时，Switch 的逻辑 Lane Number 0~3 与 Device A 的物理 Lane Number 3~0 对应。

Switch 和 Device A 都可以处理 PCIe 链路的错序连接，本节所述的实例是 Switch 支持 PCIe 链路的错序连接而 Device A 不支持这种错序连接，此时 Device A 不进行物理 Lane Number 和逻辑 Lane Number 的转换，对于 Device A 而言，物理 Lane Number 等于逻辑 Lane Number。

当 Switch 的下游端口和对端设备的上游端口离开 Configuration.Lanenum.Accept 状态后，将进入 Configuration.Complete 状态进行 Link Number 和 Lane Number 的确认。

3. Link Number 与 Lane Number 的确认

PCIe 总线使用 Configuration.Complete 和 Configuration.Idle 状态进行 Link Number 与 Lane Number 的确认。在 Configuration.Complete 状态中，PCIe 链路的两端设备将使用 TS2 序列进行 Link 与 Lane Number 的确认。在这个 TS2 序列中的 Link Number 字段为之前确定的值，而 Lane Number 字段也为已经确认的号码。

当链路两端的设备收到这些 TS2 序列后，将进一步消除 PCIe 链路不同 Lane 的漂移 (De-skew)，并设置合理的 N_FTS 的值，还有一个重要的操作是记录当前 PCIe 设备能够支持的数据传送率^①，然后进入 Configuration.Idle 状态。

PCIe 设备处于 Configuration.Idle 状态时，PCIe 链路已经设置完毕，此时将向对端至少发送 16 个 Idle 序列，当接收逻辑 RX 收到这些 Idle 序列后，将置 LinkUp 状态位为 0b1，PCIe 数据链路层将从 DL_Inactive 状态迁移到 DL_Init 状态，而物理层将进入正常工作状态 L0。

8.2.4 Recovery 状态

Recovery 状态是 LTSSM 状态机的重要状态，其复杂程度超过 Configuration 状态。该状态可以从 L0、L1 和 L0s 状态进入，当 PCIe 设备进入低功耗状态，需要进行链路重训练时，将经过 Recovery 状态之后，才能重新进入正常工作状态，第 8.3 节将讲述如何从 L1 和 L0s 状态进入 Recovery 状态。

Recovery 状态机如图 8-10 所示，该状态机由 Recovery.RcvrLock、Recovery.Speed、Recovery.RcvrCfg 和 Recovery.Idle 共四个子状态组成。

1. 从 L0 状态进入到 Recovery 状态

当 PCIe 设备工作在 L0 状态时，出现以下情况时，将进入 Recovery 状态。

(1) 更改数据传送率。

PCIe 设备需要改变数据传送率时，将进入 Recovery 状态。在 Configuration 状态中，PCIe 链路两端设备记录了“当前 PCIe 设备能够支持的数据传送率”，如果两端设备都支持大于 2.5 GT/s 的数据传送率，可以从 L0 状态进入 Recovery 状态。

在 PCIe 设备中存在两个状态位。其中 directed_speed_change 位为 1 时，表示 PCIe 设备

① 如果 PCIe 设备需要改变数据传送率时，可以在 Recovery 状态中进行更改，但是在 Configuration 状态不能进行该操作。

希望更改数据传送率，而 `changed_speed_recovery` 位为 1 时表示 PCIe 链路已经完成数据传送率的更改。

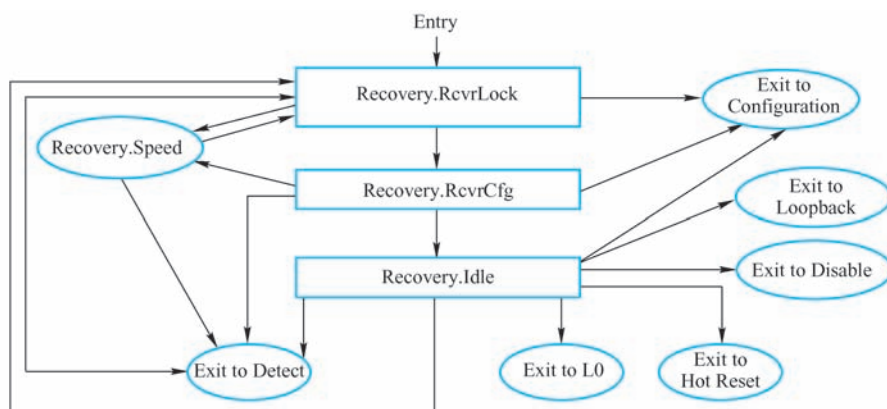


图 8-10 Recovery 状态机

当数据链路层处于 DL_Active 状态时，系统软件可以置 `directed_speed_change` 位为 1，同时复位 `changed_speed_recovery` 位为 0，随后该设备将进入 Recovery 状态。值得注意的是 PCIe 链路的两端设备需要同时改变 `directed_speed_change` 和 `changed_speed_recovery` 位。

(2) 更改链路宽度。

PCIe 设备在 Configuration 状态时，如果设置 `upconfigure_capable` 状态位为 1，可以更改链路宽度。此时该设备需要从 L0 状态进入 Recovery 状态，之后才能进行该操作。

(3) 已配置完毕的 Lane 中收到 TS1 或者 TS2 序列。

(4) 检测或者推断出对端设备处于 Idle 状态。

当 PCIe 设备检测到对端处于 Idle 状态时，将有可能进入 Recovery 状态。此处的 Idle 状态由两部分内容组成，一个通过硬件检测对端设备的发送逻辑 TX 是否处于“Electrical Idle”状态；另一个通过逻辑推断对端设备的发送逻辑是否处于 Logical Idle 状态。

在正常情况下，发送逻辑 TX 在进入 Electrical Idle 状态时，需要发送 EIOS 序列。如果对端的接收逻辑 RX 检测到 PCIe 链路实际上已经进入 Idle 状态，但是并没有收到 EIOS 序列时，将认为 PCIe 链路出现某种故障，PCIe 总线并没有规定在这种情况下，PCIe 设备是进入 Recovery 状态还是继续保持在 L0 状态中，但是在多数实现中，都将进入 Recovery 状态。

值得注意的是，一个 PCIe 设备进入 Recovery 状态之后，将向对端发送 TS1 序列，而对端设备收到这个 TS1 序列后，也将从 L0 状态进入 Recovery 状态。因此在 PCIe 链路中，当一端设备进入 Recovery 状态后，对端设备也将进入该状态。

2. Recovery.RcvrLock 状态

PCIe 设备进入 Recovery 状态时，将首先到达 Recovery.RcvrLock 状态。PCIe 设备进入该状态时将连续向对端“已配置完毕的 Lane”发送 TS1 序列，该序列的 Link 和 Lane Number 为之前在 Configuration 状态中设置的值。

如果 `directed_speed_change` 位为 1 时，发送的 TS1 序列的 `speed_change` 位也将设置为 1。PCIe 设备如果在 Recovery.RcvrLock 状态时，收到 `speed_change` 位为 1 的 TS1 序列时，该设

备的 `directed_speed_change` 位也将改变为 1。

下文以一个实例说明 `speed_change` 位和 `directed_speed_change` 位的作用。假设在一条 PCIe 链路上连接了两个设备，EP A 和 Switch 的下游端口 B，而且这两个设备的工作状态的初始值为 L0。

如果 EP A 希望进入 Recovery 状态改变数据传送率，则置 `directed_speed_change` 位为 1，随后进入 Recovery.RcvLock 状态，同时向端口 B 发送 `speed_change` 位为 1 的 TS1 序列。

而当端口 B 处于 L0 状态时，收到这个 TS1 序列后，并不会检测 `speed_change` 位，而直接进入 Recovery 状态。然后向 EP A 发送若干个 `speed_change` 位为 0 的 TS1 序列，因为此时端口 B 的 `directed_speed_change` 位为 0。

当端口 B 收到“8 个连续”的从 EP A 发来的“`speed_change` 位为 1”的 TS1 序列（还包括 EP A 支持的数据传送率种类）之后，`directed_speed_change` 位将置为 1，然后再向 EP A 发送“`speed_change` 位为 1”的 TS1 序列。值得注意的是，在上述实例中，端口 B 在 Recovery.RcvLock 状态将发送两种不同的 TS1 序列，一种序列的 `speed_change` 位为 0，而另一种序列的 `speed_change` 位为 1。

PCIe 设备连续发送 TS1 序列的同时，将通过其接收逻辑 RX 检测，是否收到 8 个连续的 TS1 或者 TS2 序列，是否这些序列使用的 Link 和 Lane Number 与该设备发送的值相同，而且这些序列的 `speed_change` 位是否与 `directed_speed_change` 位相同。如果检测成功，该设备将进入 Recovery.RcvrCfg 状态。在该状态中，PCIe 设备将重新获得 Bit/Symbol Lock，并处理不同 Lane 之间的 Skew，这也是该状态位被命名为“Lock”的原因。

PCIe 设备还可以从该状态直接进入 Recovery.Speed BFQ 状态，如果当前 PCIe 链路已经工作在大于 2.5 GT/s 的数据传送率，但是并不能稳定工作（并没有正确获得 Bit/Symbol Lock），此时需要进入 Recovery.Speed 状态，将数据传送率更改为 2.5 GT/s。PCIe 设备还可以从该状态进入 Configuration 或者 Detect 状态。本节对此不做详细介绍。

3. Recovery.RcvrCfg 状态

PCIe 设备进入 Recovery.RcvrCfg 状态后，将向对端发送 TS2 序列，该序列的 Link 和 Lane Number 为之前 in Configuration 状态设置的值。如果 `directed_speed_change` 位为 1 时，该序列的 `speed_change` 位也必须为 1。发送这些 TS2 序列的主要目的是进一步确认 Recovery.RcvrLock 状态使用 TS1 序列所获得的结果。

之后 PCIe 设备通过其接收逻辑 RX 检测，是否收到了 8 个连续的 TS2 序列，这些序列使用的 Link 和 Lane Number 与该设备发送的值是否相同，而且这些序列的 `speed_change` 位是否与 `directed_speed_change` 位相同。

如果检测成功，PCIe 设备将根据 TS2 序列中的 `speed_change` 位决定进入 Recovery.Speed 状态，还是 Recovery.Idle 状态。

如果该 PCIe 设备能够支持的数据传送率与接收的 TS2 序列中的数据传送率重合，比如都支持 5 GT/s 的数据传送率，而且 `speed_change` 位为 1 时，将进入 Recovery.Speed 状态，PCIe 设备从 Recovery.RcvrCfg 状态迁移到 Recovery.Speed 状态还有一个补充条件，即接收到第一个 TS2 序列后，至少向对端发送 32 个连续的 TS2 序列。

如果 `Speed_change` 位为 0 时，将进入 Recovery.Idle 状态，PCIe 设备从 Recovery.RcvrCfg 状态迁移到 Recovery.Idle 状态还有一个补充条件，即从接收到第一个 TS2 序列后，向对端发

送 16 个连续的 TS2 序列。

如果在 Recovery.RcvrLock 状态中, PCIe 设备没有处理不同 Lane 之间的 Skew, 在该状态中, PCIe 设备可以处理不同 Lane 之间的 Skew。PCIe 设备还可以从该状态进入 Configuration 或者 Detect 状态。本节对此不做详细介绍。

4. Recovery.Speed 状态

PCIe 设备进入该状态时, 如果使用的数据传送率为 2.5 GT/s, 则其发送逻辑 TX 需要发送 1 个 EIOS 序列, 然后进入 Electrical Idle 状态; 如果使用的数据传送率为 5.0 GT/s, 则其发送逻辑 TX 需要发送 2 个 EIOS 序列, 然后进入 Electrical Idle 状态。此时该 PCIe 设备需要等待其接收逻辑 RX 也进入 Electrical Idle 状态, 才能进一步工作。接收逻辑检测对端发送逻辑 TX 是否处于 Electrical Idle 状态, 使用的方法如第 8.1.2 节所示。

当 PCIe 设备的发送与接收逻辑都进入到 Electrical Idle 状态后, 至少需要等待 800 ns 判断 PCIe 链路两端是否成功完成数据传送率的协商, 或者等待 1 ms 判断协商是否成功。协商成功后 successful_speed_negotiation 位被置为 1, 否则将置 0。

如果协商成功, PCIe 设备将置 directed_speed_change 位为 0, 然后从该状态回到 Recovery.RcvrLock 状态。值得注意的是在 Recovery.Speed 状态并不发送 PLP 检查新的数据传送率是否能够使用。而在 Recovery.RcvrLock 状态中, 通过 TS1 序列判断对端设备是否能够获得 Bit/Symbol Lock, 确定新的数据传送率是否正常工作。如果不能正常工作, 该设备还将从 Recovery.RcvrLock 状态回到 Recovery.Speed 状态, 将数据传送率降低为 2.5 GT/s 后, 再次回到 Recovery.RcvrLock 状态。

5. Recovery.Idle 状态

PCIe 链路进入 Recovery.Idle 状态后, 将连续向对端发送 Idle 序列, 当接收逻辑 RX 的所有可用 Lane 收到 8 个连续的 Idle 序列, 而且发送逻辑 TX 发送了 16 个 Idle 序列后, 将进入 L0 状态, 完成链路训练或者重训练过程; 否则将视情况进入 Disabled、Loopback、Hot Reset、Configuration 和 Detect 状态, 本节对 Recovery.Idle 进入这些状态不做详细分析。

8.2.5 LTSSM 的其他状态

如图 8-5 所示, 在 LTSSM 中还含有 Disabled、Host Reset、Loopback、Recovery、L0、L1、L2 和 L0s 状态。其中 L0、L1、L2 和 L0s 与 PCIe 的电源管理相关, 在第 8.3 节将详细介绍这些状态的转换关系。

PCIe 链路还可以从 Configuration 和 Recovery 状态进入 Disabled 状态。当系统软件需要关闭 PCIe 链路时, 可以通过设置 Link Control 寄存器的 Link Disabled 位, 使 PCIe 链路进入 Disabled 状态。物理层进入 Disabled 状态时, 将禁止 PCIe 链路的使用, 然后视情况进入 PCIe 链路的初始状态 Detect, 重新进行 PCIe 链路的训练工作。

而 Loopback 状态是一种调试状态, PCIe 总线测试仪器可以使 PCIe 链路的对端设备进入该状态, 然后对 PCIe 链路进行测试。

Hot Reset 状态从 Recovery 状态进入, 当系统软件对 PCIe 链路进行 Hot Reset 操作时, PCIe 链路将进入该状态, 然后进入 PCIe 链路的初始状态 Detect。

8.3 PCIe 总线的 ASPM

PCIe 总线的电源管理包含 ASPM 和软件电源管理两方面内容。所谓 ASPM 是指 PCIe 链路在没有系统软件参与的情况下，由 PCIe 链路自发进行的电源管理方式。而软件电源管理指 PCI PM 机制，PCIe 总线的软件电源管理与 PCI 总线兼容。

对于一个通用处理器系统，电源管理的硬件实现与软件处理过程都较为复杂。而对于一个专用的处理器系统，如手机应用，在多数情况下，更侧重于在某些用户场景中，功耗的使用情况，其设计难度相对较小。而无论是对于通用还是专用处理器系统，电源管理都需要处理器与外部设备的参与，协调完成。

PCIe 总线为 PCIe 设备提供了几种低功耗模式。在一个处理器系统中，PCIe 设备的低功耗模式需要与处理器的低功耗模式协调工作，以最优化整个处理器系统的功耗。

对于某些专用的处理器系统，外部设备之间也需要协调工作，以最优化整个处理器系统的使用的功耗。目前电源管理已经成为处理器系统实现的一个热点。但是 PCI/PCIe 总线提供的电源管理机制远非完美，该管理机制仅考虑了外部设备与处理器系统之间的电源使用关系，并没有考虑外部设备之间电源的使用关系。

在一个处理器系统中，外部设备越来越智能化，处理器与外设间的通信基本等同于两个处理器间的通信，适用于这类处理器系统的电源管理机制也有待研究。在有些处理器系统中，专门设置了用于电源管理的微处理器，协调“主处理器”与“外部设备”及外部设备间的电源使用情况，以最优化整个处理器系统的电源管理。目前在智能手机的设计中，通常具有专门用于电源管理的微处理器。

8.3.1 与电源管理相关的链路状态

PCIe 总线定义了一系列与电源管理相关的链路状态。

- L0 状态。PCIe 设备的正常工作状态。
- L0s 状态。PCIe 设备处于低功耗状态。系统软件不能控制 L0 状态和 L0s 状态间的迁移过程，这两个状态的迁移只能由 ASPM 控制。
- L1 状态。PCIe 设备使用的功耗低于处于 L0s 状态时的功耗。
- L2/L3 Ready 状态。PCI 设备进入 L2 或者 L3 状态之前使用的过渡状态。
- L2 状态。PCIe 设备仅使用辅助电源工作，主电源已经被关闭。在 PCIe 总线中 L1 和 L2 状态是可选的。
- L3 状态。该状态也被称为“Link off”状态，此时 PCIe 设备使用的 V_{cc} 电源已经被关闭。
- LDn 状态。该状态是一个“伪”状态，PCIe 链路处于 L2、L3 状态时，需要通过 LDn 状态之后才能进入 L0 状态。该状态由 LTSSM 状态机的 Detect、Polling 和 Configuration 等状态组成。

这些与电源管理相关的状态机迁移模型如图 8-11 所示。

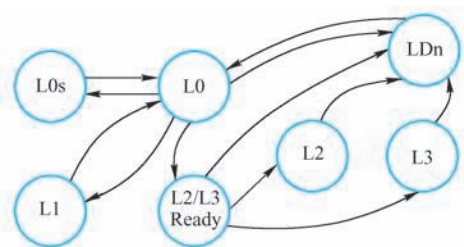


图 8-11 电源管理状态机

本节重点说明 L0、L0s 和 L1 状态的工作原理以及如何使用 ASPM 机制进行状态迁移。在第 8.4 节将讲述系统软件如何设置寄存器使 PCIe 设备进入 L0、L0s 和 L1 状态。

在 PCIe 设备中，Link Capabilities 寄存器的 ASPM Support 字段表示当前 PCIe 设备可以支持的链路状态，该字段只读。而 Link Control 寄存器的 ASPM Control 字段为可读写的，PCIe 设备根据 ASPM Support 字段判断当前 PCIe 链路是否支持 L0s 和 L1 状态，还是同时支持这两种状态，并设置 ASPM Control 字段。

8.3.2 L0 状态

PCIe 设备从 Configuration、Recovery 和 L0s 状态进入 L0 状态时，L0 状态是 PCIe 设备的正常工作状态。此时 PCIe 设备可以通过 PCIe 链路，发送和接收 TLP、DLLP 和 PLP，此时 LinkUp 状态位为 1，而数据链路层处于 DL_Active 状态。PCIe 设备从 L0 状态可以进入 Recovery、L0s、L1 和 L2/L3 Ready 状态，本节重点讨论从 L0 状态进入 Recovery、L0s 和 L1 状态的情况。而从 L0 状态进入 Recovery 状态的条件见第 8.2.4 节。

1. 进入 L0s 状态

当 PCIe 设备发现链路“空闲”时^①，可以主动进入 L0s 状态。RC、EP 和 Switch 进入 L0s 状态的“空闲”条件并不相同。对于含有多个 Function 的上游端口，只有所有 Function 都处于“空闲”状态时，才被认为“空闲”。此处“空闲”的定义与 LTSSM 的 Electrical Idle 和 Logical Idle 间没有任何联系。

对于 RC 或者 EP 的端口，当以下两个条件同时满足时，当前端口被认为是“空闲”的。

- 1) 没有准备发送的 TLP，或者对端没有提供足够的 Credit^②。
- 2) 没有准备发送的 DLLP。

对于 Switch 的上游端口，当以下三个条件同时满足时，发送逻辑 TX 认为 PCIe 链路是“空闲”的。

- 1) Switch 所有下游端口的接收链路处于 L0s 状态。
- 2) 没有准备发送的 TLP，或者对端没有提供足够的 Credit。
- 3) 没有准备发送的 DLLP。

① 当空闲时间不大于 7 μ s 时，将进入 L0s 状态，但是 PCIe 总线并没有规定进入 L0s 状态的最小时间。

② 对端没有提供足够的 Credit 时，发送端不能发送 TLP。

对于 Switch 的下游端口，当以下三个条件同时满足时，发送逻辑 TX 认为 PCIe 链路是“空闲”的。

- 1) Switch 的所有上游端口的接收链路处于 L0s 状态。
- 2) 没有准备发送的 TLP，或者对端没有提供足够的 Credit。
- 3) 没有准备发送的 DLLP。

值得注意的是，一个接收端或者发送端的发送逻辑 TX 和接收逻辑 RX 在同一时刻，可能处于不同的 LTSSM 状态，一个处于 L0，而另一个处于 L0s。

2. 进入 L1 状态和 L2 状态

PCIe 设备可以通过上层软件，将链路状态从 L0 状态迁移到 L1 或者 L2 状态。当 PCIe 设备进入 D1~D3 状态时，其上游链路将进入 L1 状态；而进入 D3_{Cold} 状态时，其上游链路将进入 L2 状态。D1~D3 和 D3_{Cold} 状态的详细说明见第 8.4.1 节。而 PCIe 链路的两端设备需要同时进入 L1 或者 L2 状态。

首先两端设备的发送逻辑 TX 分别向对端发送 EIOS 序列，随后发送逻辑 TX 进入 Electrical Idle 状态。如果 PCIe 设备的接收逻辑 RX 从任意 Lane 中收到 1 个或者 2 个 EIOS 序列后，该设备将进入到 L1 或者 L2 状态。当 PCIe 链路工作在 2.5 GT/s 时，需要发送 1 个 EIOS 序列；如果工作在 5 GT/s 时，需要发送 2 个 EIOS 序列。

值得注意的是，PCIe 设备处于 L1 状态和 L2 状态时，D+和 D-信号输出的 DC 共模电压并不相同。处于 L1 状态时，PCIe 设备的发送逻辑 TX 仍然需要维持一个相对较低的 DC 共模电压，其伏值比其处于 L0 状态时低 $V_{TX-CM-DC-ACTIVE-IDLE-DELTA}$ （最小值为 0，最大值为 100 mV），如式（8-1）所示。

$$|V_{TX-CM-CD[During L0]} - V_{TX-CM-Idle-DC[During Electrical Idle]}| \leq 100 \text{ mV} \quad (8-1)$$

而 PCIe 设备在 L2 状态时，其发送逻辑 TX 并没有这种限制，而且其发送逻辑和接收逻辑基本处于下电状态。这正是 PCIe 设备处于 L2 状态时使用的功耗低于 L1 状态的原因，同时也是从 L2 状态进入 L0 状态，更加耗时的原因。

如果 PCIe 设备支持 Beacon 机制，那么处于 L2 状态时，PCIe 设备的发送逻辑 TX 能够发送 Beacon 信号，而接收逻辑 RX 能够检测 Beacon 信号^①。Beacon 机制是一种唤醒机制。当 PCIe 设备处于 L2 状态时，可以使用该机制唤醒。

8.3.3 L0s 状态

PCIe 设备必须支持 L0s 状态。L0s 状态是一个低功耗状态，PCIe 设备进入或者退出该状态不需要系统软件的干预，其状态转换由硬件控制完成。L0s 的状态转换由两部分组成，一个是接收状态机，另一个是发送状态机。

同一个 PCIe 设备的发送逻辑 TX 和接收逻辑 RX，在同一时刻可能处于不同的链路状态，其中一个为 L0，而另一个为 L0s。例如当一个 EP 进行 DMA 写操作时，其发送逻辑 TX 一直被使用，因此处于 L0 状态，而接收逻辑 RX 可能长时间没有被使用，从而可以暂时处

① Beacon 信号并不是实际信号，而是通过 D+和 D-信号发送的一个频率在 30 KHz~500 MHz 之间的脉冲信号，该信号可以唤醒 PCIe 设备。在许多 PCIe 设备中，并不含有 WAKE#信号，此时需要使用 Beacon 机制唤醒 PCIe 设备。

于 L0s 状态，以降低功耗。

1. 发送逻辑 TX 状态机

L0s 的发送状态机如图 8-12 所示，该状态机由 TX_L0s.Entry、TX_L0s.Idle 和 TX_L0s.FTS 状态组成。



图 8-12 L0s 的发送状态机

PCIe 设备处于 L0 状态发现链路为临时“空闲”状态时，将进入 TX_L0s.Entry 状态。处于该状态时，发送逻辑 TX 首先向对端发送 1 或者 2 个 EIOS 序列^①，之后进入 Electrical Idle 状态。再经过 20 ns 延时后，发送逻辑 TX 进入 TX_L0s.Idle 状态。

当发送逻辑 TX 处于 TX_L0s.Idle 状态时，如果 PCIe 设备需要发送数据报文，发送逻辑 TX 将退出 TX_L0s.Idle 状态，进入 TX_L0s.FTS 状态。发送逻辑 TX 处于 TX_L0s.FTS 状态时，向对端顺序发送 N_FTS 个 FTS 序列^②和 1 个 SKP 序列之后，将进入 L0 状态。

2. 接收逻辑 RX 状态机

L0s 的接收状态机如图 8-13 所示，该状态机由 RX_L0s.Entry、RX_L0s.Idle 和 RX_L0s.FTS 状态组成。PCIe 设备可以从 L0s 状态进入 L0 或者 Recovery 状态。

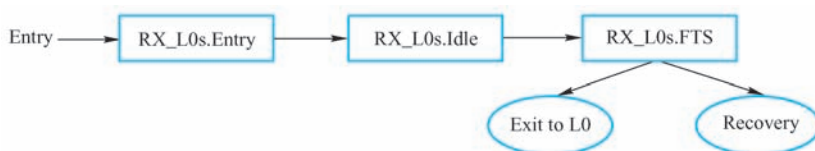


图 8-13 L0s 的接收状态机

接收逻辑 RX 处于 L0 状态时，如果收到 1 个 EIOS 序列后，将进入 RX_L0s.Entry 状态。接收逻辑 RX 在 RX_L0s.Entry 状态经过一段延时^③后，将进入 RX_L0s.Idle 状态。

接收逻辑 RX 在 RX_L0s.Idle 状态中将持续监测接收链路，如果发现对端设备的发送逻辑 TX 退出“Electrical Idle”状态时，接收逻辑 RX 将进入 RX_L0s.FTS 状态。

当接收逻辑 RX 处于 RX_L0s.FTS 状态时，PCIe 链路的每一个 Lane 都将收到 N_FTS 个 FTS 序列，接收逻辑 RX 使用这些 FTS 序列重新获得 Bit/Symbol Lock。如果对端发送逻辑 TX 发送的 FTS 序列不足，接收逻辑 RX 将无法成功获得 Bit/Symbol Lock，此时 PCIe 设备将进入 Recovery 状态。

当接收逻辑 RX 收到足够数量的 FTS 序列，又收到了一个 SKP 序列后（该 SKP 序列的作用是 De-Skew），将从 RX_L0s.FTS 状态迁移到 L0 状态。

8.3.4 L1 状态

L1 状态是一个比 L0s 状态使用功耗更低的状态，PCIe 设备从 L1 状态恢复到 L0 状态，比 L0s 状态恢复到 L0 状态的延时更长。PCIe 设备进入或者退出该状态可以不需要系统软件

① 当前 PCIe 链路为 2.5 GT/s 时发送 1 个 EIOS 序列，否则发送 2 个 EIOS 序列。

② 如果 Link Control 寄存器的 Extended Sync 位为 1 时，发送部件将向对端发送 4096 个 FTS 序列。

③ PCIe 规范规定这段延时为 $T_{TX-IDLE-MIN}$ （最小值为 20 ns）。

的干预。当然系统软件也可以通过设置某些寄存器，使 PCIe 链路的两端设备同时进入 L1 状态。在 PCIe 总线中，L1 状态是一个可选状态。

其中只有下游设备（EP 或者 Switch 的上游端口）可以主动“进入 L1 状态”，而上游设备（RC 或者 Switch 的下游端口）必须与下游设备进行协商后才能进入 L1 状态。当下游设备满足以下条件时，可以进入 L1 状态。

- 1) PCIe 设备支持 L1 状态。
- 2) PCIe 设备没有准备发送的 TLP 和 DLLP。
- 3) 如果下游设备是一个 Switch，这个 Switch 的所有下游端口处于 L1 或者更高一级的节电状态。

而上游设备需要经过协商才能进入 L1 状态。

- 1) 首先下游设备向上游设备发送 PM_Active_State_Request_L1 报文。
- 2) 上游设备收到这个 DLLP 报文后，如果该上游设备可以进入 L1 状态，则向下游设备发送 PM_Request_Ack 报文；如果不能进入，则发送 PM_Active_State_Nak 报文。

在 PCIe 总线中，L1 状态机由 L1.Entry 和 L1.Idle 两个子状态组成，如图 8-14 所示。



图 8-14 L1 状态机

PCIe 设备从 L0 状态首先进入 L1.Entry 状态。PCIe 设备处于 L1.Entry 状态时，发送逻辑 TX 处于 Electrical Idle 状态。PCIe 设备在此状态停留 20 ns 后，进入 L1.Idle 状态。接收逻辑在 L1.Idle 状态中将持续监测接收链路，如果发现其对端发送逻辑 TX 退出“Electrical Idle”状态时，将从 L1.Idle 状态首先迁移到 Recovery 状态，而不是 L0 状态。

PCIe 设备处于该状态时，其发送逻辑 TX 可以处于高阻抗或者低阻抗模式，而其接收逻辑 RX 必须处于低阻抗模式。

8.3.5 L2 状态

当 PCIe 设备处于 L2 状态时，使用的功耗低于 L1 状态，但是恢复到 L0 状态的延时更长。当 PCIe 设备处于 L2 状态时，需要首先迁移到 Detect 状态，重新进行链路训练。L2 状态是一个可选状态。L2 状态机由 L2.Idle 和 L2.TransmitWake 两个子状态组成，如图 8-15 所示。



图 8-15 L2 状态机

在 L2.Idle 状态中，接收逻辑 RX 的端接必须处于低阻抗模式，而发送逻辑 TX 必须在 Electrical Idle 状态中至少停留 20 ns。当一个 EP、Switch 或者 RC 的某个端口被唤醒后，将首先从 L2.Idle 状态迁移到 L2.TransmitWake 状态。

PCIe 设备进入 L2.TransmitWake 状态后，将向 RC 端口或者 Switch 的下游端口发送 Beacon 信号。当 RC 端口收到这个 Beacon 信号后，将进入 Detect 状态进行链路训练；而当 Switch 收到 Beacon 信号被唤醒后，其上游端口将进入 L2.TransmitWake 状态，并向上游链路转发这个 Beacon 信号，并逐级唤醒 PCIe 链路的上游设备。

当 EP 或者 Switch 上游端口的接收逻辑 RX，发现其对端发送逻辑 TX 退出 Electrical Idle

状态时，将从 L2.TransmitWake 状态迁移到 Detect 状态，重新进行链路训练。

8.4 PCI PM 机制

PCIe 总线与 PCI 总线使用的 PCI-PM 管理机制兼容。在 PCIe 设备的扩展配置空间中定义了 Power Management Capabilities 结构，该结构中含有一系列寄存器，这些寄存器的详细说明见第 4.3.1 节。

系统软件通过修改 PMCSR 寄存器的 Power State 字段，可以使 PCIe 设备进入不同的节能状态 D-State，如 D0、D1、D2 和 D3 状态。其中 D0 是正常工作状态，功耗最高，而 D1、D2 和 D3 为低功耗状态。其中 D1 的休眠等级最低，功耗相对较高，而 D3 的休眠等级最高，功耗相对较低。D-State 的状态转换关系如图 8-16 所示。

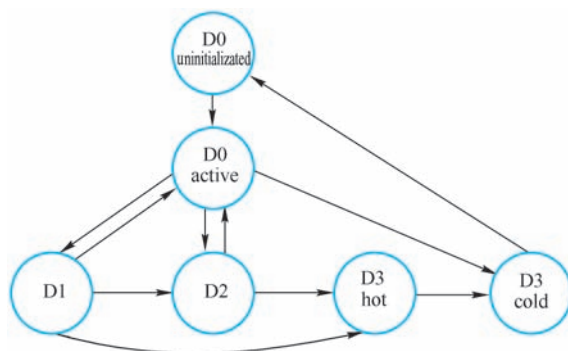


图 8-16 D-State 状态机

值得注意的是，当 PCIe 设备进行状态迁移时，PCIe 链路也需要视情况进入相应的状态，如进入 L1 或者 L2 等状态。

8.4.1 PCIe 设备的 D-State

PCIe 设备的 D-State 由 D0、D1、D2 和 D3 状态组成。其中 D0 状态由 D0_{uninitialized} 和 D0_{active} 两个子状态组成，而 D3 状态由 D3_{hot} 和 D3_{cold} 两个子状态组成。

1. D0 状态

PCIe 设备必须支持 D0 状态，该状态由 D0_{uninitialized} 和 D0_{active} 两个子状态组成。当 PCIe 设备处于 D0_{uninitialized} 状态时，该 PCIe 设备并没有被系统软件使能，此时该 PCIe 设备仅能接收配置读写请求 TLP，不能主动发出其他 TLP。此时该 PCIe 设备配置寄存器的 Command 寄存器为复位值 0x00。此时虽然 PCIe 设备已经被加电，但是并不能正常使用。

当 PCIe 设备处于 D0_{active} 状态时，PCIe 设备处于正常工作模式，并没有任何节电措施。但是 PCIe 设备仍然可以使用 ASPM 机制，将链路状态迁移到 L0s 或者 L1 状态，以降低功耗。值得注意的是，ASPM 机制与 PCI PM 机制是独立的。

当 PCIe 设备进行复位后，该设备将首先进入 D0_{uninitialized} 状态。系统软件通过修改 PMCSR 寄存器的 Power State 字段，也可以使设备从 D3_{hot} 状态迁移到该状态。值得注意的是 D3_{cold} 状态迁移到该状态的过程与复位操作等效。当系统软件改写 Command 寄存器的状态位

使能 PCIe 设备后, 该设备从 $D0_{uninitialized}$ 迁移到 $D0_{active}$ 状态。

2. D1 和 D2 状态

D1 和 D2 状态分别为 PCIe 设备的轻度和重度休眠状态。这两个状态为 PCIe 设备的可选状态, PCIe 设备处于 D1 状态时的功耗高于 D2 状态。

PCIe 设备处于这两个状态时, 除了 PME 消息之外, 不能主动发送其他 TLP; 除了接收配置请求 TLP 外, 不能接收其他 TLP。当 PCIe 设备处于这两种状态时, 可以向 RC 发送 PME 消息, 通知系统软件该 PCIe 设备进入休眠状态。当 PCIe 设备进入 D1 或者 D2 状态时, PCIe 链路将进入 L1 状态。PCIe 设备可以从 D1 和 D2 状态直接返回到 $D0_{active}$ 状态。

3. D3 状态

PCIe 设备必须支持 D3 状态, D3 状态由 $D3_{hot}$ 和 $D3_{cold}$ 两个子状态组成。PCIe 设备处于 $D3_{hot}$ 状态与处于 D1/D2 状态时的功能类似, 只是 PCIe 设备只能从 $D3_{hot}$ 状态返回 $D0_{uninitialized}$ 状态, 而不能返回 $D0_{active}$ 状态。对于 PCIe 设备, 从 $D3_{hot}$ 状态返回 $D0_{uninitialized}$ 状态的过程相当于热复位。

当 PCIe 设备的 V_{cc} 电源被移除时, PCIe 设备无论处于何种状态, 都将进入 $D3_{cold}$ 状态。值得注意的是一个 PCIe 设备使用两种电源 V_{cc} 和 V_{aux} , V_{cc} 电源被移除并不意味着 PCIe 设备被完全下电。

有些 PCIe 设备在处于 $D3_{cold}$ 状态时仍然可以发出 PME 消息, 此时这个 PCIe 设备负责发送 PME 消息的功能模块必须使用 V_{aux} 而不是 V_{cc} 进行供电。

8.4.2 D-State 的状态迁移

如图 8-16 所示, PCIe 设备可以进行 D-State 的状态迁移。大多数 D-State 的状态迁移都是系统软件通过修改 PMCSR 寄存器的 Power State 字段实现的, 但是仍然有些状态迁移采用了其他方式。

- 使能 Command 寄存器的命令位, 可以使设备从 $D0_{uninitialized}$ 状态迁移到 $D0_{active}$ 状态。
- PCIe 设备的 V_{cc} 被移除时, $D3_{hot}$ 状态将迁移到 $D3_{cold}$ 状态。
- 当 PCIe 被唤醒, V_{cc} 重新上电之后, PCIe 设备将从 $D3_{cold}$ 状态迁移到 $D0_{uninitialized}$ 状态。

当 PCIe 设备进行 D-State 状态迁移时, PCIe 链路的状态也可能随之变化。PCIe 设备的 D-State 状态与 PCIe 链路状态的对应关系如表 8-3 所示。

表 8-3 D-State 状态与 PCIe 链路状态的对应关系

下游设备的 D-State	上游设备可能的 D-State	可能的链路状态
D0	D0	L0, L0s, L1, L2/L3 Ready
D1	D0~D1	L1, L2/L3 Ready
D2	D0~D2	L1, L2/L3 Ready
$D3_{hot}$	D0~ $D3_{hot}$	L1, L2/L3 Ready
$D3_{cold}$	D0~ $D3_{cold}$	L2, L3

由上表可以发现, 上游设备所处的 D-State 等级小于或等于下游设备的休眠等级。如下游设备处于 D1 状态时, 上游设备不能处于比 D1 更高的休眠等级, 如 D2 或者 D3 状态。

当设备处于 $D0 \sim D3_{cold}$ 状态时, ASPM 机制可以根据链路的使用情况进行链路状态的迁移, 而无需软件的干预。下文以 PCIe 设备从 D0 迁移到 D1 说明 D-State 进行状态迁移时,

PCIe 链路如何进行状态迁移。

当系统软件修改 PMCSR 寄存器的 Power State 字段，将 PCIe 设备从 D0 迁移到 D1 状态时，上游设备与下游设备将协调工作，完成 PCIe 设备的状态切换，并改变链路的状态，其实现过程如图 8-17 所示。

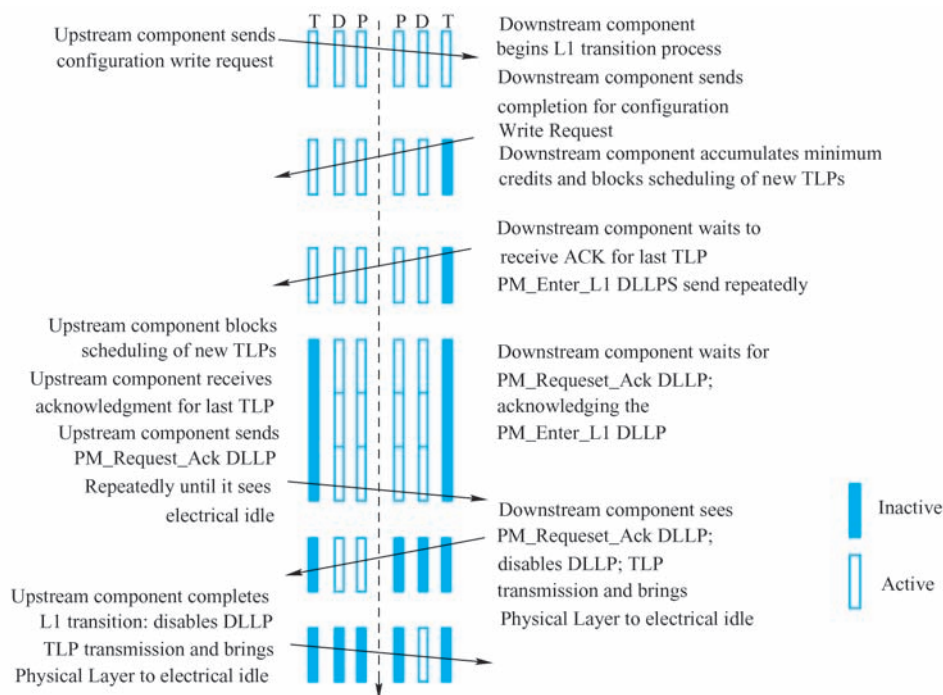


图 8-17 PCIe 设备从 D0 到 D1 的状态迁移

1) 上游设备向下游设备发送配置写请求，改变下游设备 PMCSR 寄存器的 Power State 字段，从而使下游设备从 D0 状态迁移到 D1 状态。

2) 下游设备收到这个配置写请求 TLP 后，将改变 PMCSR 寄存器的 Power State 字段，并向上游设备发送配置写完成 TLP。这个配置写完成 TLP 首先需要经过数据链路层，并从对端获得足够的发送 Credit^①后，将这个配置写完成 TLP 通过数据链路层发送到对端。

3) 下游设备的事务层收到数据链路层的确认后，得知配置写完成 TLP 已经被上游设备正确接收后（详见 ACK/NAK 协议），将挂起下游设备的事务层。并向上游设备连续发送 PM_Enter_L1 DLLP，同时等待来自上游设备的 PM_Request_Ack 报文。

4) 上游设备收到下游设备的 PM_Enter_L1 DLLP 后，首先禁止发送新的 TLP，并等待之前发送的 Non-Post TLP 得到确认后，挂起上游设备的事务层，并向下游设备连续发送 PM_Request_Ack DLLP。

5) 下游设备在没有收到上游设备的 PM_Request_Ack DLLP 之前，虽然事务层已经被挂起，但是数据链路层和物理层仍然可以正常工作，此时数据链路层可以正确接收来自上游端

① 发送 Credit 的详细说明见第 9 章流量控制。

口的 DLLP，并发送 ACK/NAK 和流量控制相关的一些 DLLP。

6) 当下游设备收到 PM_Request_Ack DLLP 后，将停止发送 PM_Enter_L1 DLLP，挂起数据链路层，然后将物理层置为 Electrical Idle 状态。

7) 上游设备发现其接收链路处于 Electrical Idle 状态时，将停止发送 PM_Request_Ack DLLP，并挂起数据链路层，然后将物理层置为 Electrical Idle 状态。此时 PCIe 链路将进入 L1 状态。

当 PCIe 链路处于 L1 状态时，如果系统软件需要改变下游 PCIe 设备 PMCSR 寄存器的 Power State 字段，PCIe 链路需要首先从 L1 状态迁移到正常工作状态 L0，下游设备才能接收这个配置写请求 TLP。

PCIe 设备其他 D-State 状态的迁移过程与此大同小异，详见 PCIe 总线规范，本节对此不做进一步描述。

8.5 小结

本章重点介绍 PCIe 总线的 LTSSM 状态机，该状态机的迁移模型较为复杂。本章仅介绍了该状态机的基本工作路径，对此部分有兴趣的读者可以阅读 PCIe 总线规范，进一步了解相关内容。

LTSSM 状态机在 PCIe 总线规范中处于核心地位，深入理解该状态机的运转模型，有利于底层软件工程师深入理解 PCIe 设备的工作状态，从而开发出质量较高的程序。

本章还使用一定篇幅介绍了 PCIe 总线的电源管理模型。目前电源管理已经成为计算机体系结构的热点。一个合理的电源管理模型需要软硬件的共同参与，但是硬件设计仍然决定了电源管理模型的节电效率。